國立臺灣科技大學
電 子 工 程 系

碩士學位論文

應用於 nFAPI 分離架構之吞吐量—延遲權衡的自適應時序控制

Adaptive Timing Control for Throughput—Latency Trade-off in nFAPI Split

研 究 生： 徐銘鴻
學　　號： M11302209

指導教授 :鄭瑞光博士

中華民國一百一十五年六月一十五日

Recommendation Letter

M11302209

碩士學位論文指導教授推薦書
Master's Thesis Recommendation Form

系所： Department/Graduate Institute	電子工程系 Department of Electronic and Computer Engineering
姓名： Name	徐銘鴻 MING HONG HSU
論文題目： (Thesis Title)	應用於 nFAPI 分離架構之吞吐量－延遲權衡的自適應時序控制 Adaptive Timing Control for Throughput–Latency Trade-off in nFAPI Split

係由本人指導撰述，同意提付審查。
This is to certify that the thesis submitted by the student named above, has been written under my supervision. I hereby approve this thesis to be applied for examination.

指導教授簽章：
Advisor's Signature 鄭瑞光

共同指導教授簽章（如有）：
Co-advisor's Signature (if any) ____________

日期：
Date(yyyy/mm/dd) 2026 / 06 / 15

Approval Letter

M11302209

碩士學位考試委員審定書

Qualification Form by Master's Degree Examination Committee

系所：
Department/Graduate Institute
電子工程系
Department of Electronic and Computer Engineering

姓名：
Name
徐銘鴻
MING HONG HSU

論文題目：
(Thesis Title)
應用於 nFAPI 分離架構之吞吐量－延遲權衡的自適應時序控制
Adaptive Timing Control for Throughput–Latency Trade-off in nFAPI Split

經本委員會審定通過，特此證明。
This is to certify that the thesis submitted by the student named above, is qualified and approved by the Examination Committee.

學位考試委員會
Degree Examination Committee

委員簽章：
Member's Signatures

蘇瑞元　曹孝櫟
[illegible]　王瑞堂
許瀚聰

指導教授簽章：
Advisor's Signature
蘇瑞元

共同指導教授簽章（如有）：
Co-advisor's Signature (if any)

系所（學程）主任（所長）簽章：
Department/Study Program/Graduate Institute Chair's Signature
林淵翔

日期：
Date(yyyy/mm/dd)
115 / 6 / 15

摘　要

在基於 nFAPI（Option 6）的拆分式無線接取網路（RAN）架構中，將媒體存取控制層（MAC Layer）與實體（PHY Layer）層分別部署於虛擬化網路功能（VNF）與實體網路功能（PNF）上，會因封包交換網路傳輸而引入關鍵的時序同步問題。為避免控制訊息因超出 PNF 的嚴格時序視窗而被丟棄，VNF 必須提前排程並發送控制訊息。本論文提出一套自適應時序控制框架，利用 PNF 即時回報的時序資訊（Timing Info）建立閉迴路回饋機制，動態進行自適應時序控制（Adaptive Timing Control）。透過持續微調發送時序，此方法能自動適應多樣的部署環境，包括動態變化的網路負載（offered load）、傳輸延遲與網路抖動（jitter）。在整合商用 Radio Unit（O-RU）與 OpenAirInterface（OAI）的實體測試平台驗證下，本機制能有效大幅減少時槽遺失錯誤、穩定來回延遲（RTT），並在嚴重網路壅塞下維持系統吞吐量之穩定。

關鍵字：nFAPI、RAN 拆分、時序同步、自適應時序控制、網路負載適應、O-RU、OAI

Abstract

nFAPI-based Split Option 6 separates the MAC scheduler and High-PHY across packet-switched transport, where scheduling messages must arrive at the PNF within a strict timing window. Because existing static scheduling relies on a fixed ahead time to trigger the scheduler, it cannot automatically adapt to varying network latency and jitter across different deployments. This thesis proposes an adaptive timing control framework for nFAPI Split 6 that dynamically adjusts the scheduler trigger offset using PNF timing info feedback. The proposed method applies EWMA-based Timing Info and jitter estimation to perform adaptive timing control, adjusting the scheduling advance rapidly under timing risk and recovering it safely under stable conditions. We implement the framework in an OpenAirInterface-based end-to-end testbed with a commercial O-RU and COTS UE. Experimental results show that the proposed method maintains stable throughput under injected delay and jitter while avoiding unnecessary MAC HARQ RTT increase compared with static scheduling baselines. Additionally, the proposed method maintains stable system-level throughput under varying offered loads in a realistic multi-hop campus deployment.

Key Words: nFAPI, RAN disaggregation, timing synchronization, adaptive timing control, network load adaptation, O-RU, OAI

Acknowledgements

首先，我想要最優先感謝我的指導教授鄭瑞光教授，感謝他在我研究過程中提供的指導和支持，並提供了非常豐富的資源給予我們，從大二暑假開始加入實驗室做專題，就能體會到老師的用心。一路提拔我們同屆的幾位同學包括我，送我們到法國 EURECOM 實習六個月，體驗了許多同齡人難以體驗的經驗，碩一暑假還讓我到聯發科實習（在此要一併感謝 JoJo、Alan、Nick 學長們的照顧），也順利拿到研發替代役兼預聘書，總之，老師的專業知識和耐心指導對我的研究工作有著重要的影響。

接著我要特別感謝我的爸爸、媽媽、妹妹還有阿公，每次回家都會有充電治癒的感覺，讓我覺得回家是充滿溫暖且令人期待的一件事情，謝謝爸爸媽媽不論是在心理上或是經濟上都一路支持我到碩士畢業，經濟的部分，等我寫完這份論文畢業後，就換我來支持你們了。

此外，我還要感謝我的同學們，特別是賴俊凱 (愷)（Kenny）和楊智元（Tobby）每天沒日沒夜的陪伴與幫助，一起討論研究問題、挑戰和分享想法。我知道每個研究生的時間都很寶貴，但你們還是願意花時間和我一起討論論文中遇到的困難與挑戰，讓我的研究過程更加扎實和有趣。還有每天一起煩惱公館哪家好吃的大家（巴部 babu、Johnson、Winnie、Joanna、Leo），之後有機會還要繼續常常一起吃飯。

當然還有實驗室的其他夥伴們（Sheryl、Will、Mira、Joshevan、Ian、Ravi、Richart），我不會忘記你們的。偶爾遇到問題大家互相幫忙，謝謝你們讓我深深體會到我們 BMW Lab 是一個 team。還有遠在法國 EURECOM 的 Robert，即便在我從 EURECOM 實習回台後，仍然持續替我解惑，給予方向與支持。

最後，我想要感謝我的女朋友，謝謝妳一直以來的支持、開導與陪

伴，讓我在面對不同的困難與挑戰時都能保有動力持續前進，謝謝妳一直陪伴在我的身邊。

總之，感謝所有在我研究過程中給予支持和幫助的人，沒有你們的支持，我無法完成這項研究。謝謝你們！

Contents

List of Figures

List of Tables

Chapter 1 Introduction

1.1 Research Background and O-RAN Architecture Evolution

In the architectural evolution of the fifth-generation (5G) and the upcoming sixth-generation (6G) mobile communication standards, the Radio Access Network (RAN) is undergoing a fundamental transformation from the traditional monolithic base station to a highly virtualized, cloudified, and disaggregated architecture. Under the traditional architecture, baseband processing and Radio Frequency (RF) functions were tightly bound to the special hardware equipment of a single vendor, which not only limited the flexibility of network deployment but also significantly increased the capital and operational expenditures of operators.

With the rise of Cloud-RAN (C-RAN), disaggregated deployment methods have gradually become popular, centralizing the Baseband Processing Unit (BBU) into a remote BBU Pool while leaving the Remote Radio Head (RRH) at the edge site, connecting the two via the Common Public Radio Interface (CPRI). However, the C-RAN architecture is still limited by the massive fronthaul bandwidth requirements and strict latency constraints. To overcome these bottlenecks and completely break the dilemma of vendor lock-in, the O-RAN Alliance [1] and the 3rd Generation Partnership Project (3GPP) jointly promoted further disaggregation of the RAN, formally splitting the traditional BBU and RRH logically and physically into three independent network entities: the Central Unit (CU), Distributed Unit

(DU), and Radio Unit (RU) [2,3].

In this disaggregated model, functional split options have become the most critical design core for determining network performance, hardware complexity, bandwidth consumption, and latency tolerance [4,5]. 3GPP has standardized a variety of functional split options to adapt to diverse network requirements and deployment scenarios.

1.1.1 Split Option 2

Split Option 2 is a High Layer Split (HLS) architecture that primarily separates the CU and DU. Under this split, higher-layer protocols such as the Packet Data Convergence Protocol (PDCP) and Radio Resource Control (RRC) run centrally in the CU, while lower-layer Radio Link Control (RLC), Medium Access Control (MAC), and Physical Layer (PHY) remain in the DU. This architecture has relatively loose fronthaul bandwidth and latency requirements, making it easy to deploy over general packet-switched networks. The design of its synchronization mechanism is also very simple; due to its high tolerance, it only relies on flow control and deep buffering to operate stably [4], and is widely recognized as the standard configuration with the best cost-effectiveness ratio, especially suitable for non-real-time data transmission scenarios.

1.1.2 Split Option 6

Split Option 6, also known as the MAC-PHY split, precisely places the architectural separation point between the MAC layer and the PHY layer.

In this option, the DU is responsible for processing upper-layer protocols including the MAC layer, while the RU retains complete physical layer (PHY) processing capabilities. Since the fronthaul network transmits MAC Protocol Data Units (MAC PDUs) scheduled by the MAC layer, its bandwidth requirements are dynamically adjusted based on the actual load of the users. Notably, although the nFAPI-based Split Option 6 architecture also supports deployment over general packet-switched networks, its synchronization requirements are completely different from Split 2. It must strictly follow the scheduler's synchronization baseline at every slot level, ensuring data is delivered precisely on time from the VNF to the PNF. During this process, the operating system scheduling delays on the VNF side, as well as the unknown transmission delays and jitter introduced by the packet-switched network, are major challenges that must be overcome to maintain precise time synchronization [6].

1.1.3 Split Option 7.2

Split Option 7.2x (Low-PHY split) places the split point inside the physical layer, and is currently the primary Lower Layer Split (LLS) standard promoted by the O-RAN Alliance [7]. Although moving partial computations (such as FFT/IFFT) to the RU alleviates fronthaul pressure, its fronthaul transmits frequency-domain IQ symbols processed by the physical layer, demanding extremely high bandwidth. Because of this, Split 7.2 must be deployed under very strict network conditions, and its reliance on hardware equipment (such as hardware-level Precision Time Protocol, PTP) results in extremely high deployment costs [8]. However, precisely because it has a strict private network environment and transmits a stable, continuous stream of IQ data, its synchronization mechanism is relatively stable and does not require special handling for latency variation issues caused by different offered loads.

1.1.4 Split Option 8

Split Option 8 is the most traditional lower-layer split architecture, placing the split point between the Radio Frequency (RF) and the Physical Layer (PHY). Under this architecture, all baseband processing functions (including complete PHY, MAC, and higher-layer protocols) are centralized in the DU or CU, while the RU is only responsible for RF signal transmission and reception. This configuration produces a fronthaul data stream dominated by time-domain IQ samples, usually relying on CPRI or enhanced CPRI (eCPRI) for transmission. Although it highly centralizes hardware resources, its fronthaul network bandwidth requirements are extremely high, and it is strictly constrained by system bandwidth and antenna count. Furthermore, its latency limits are extremely strict (maximum allowable one-way latency is $250\ \mu$s [2]), thus, similar to Split 7.2, it heavily relies on high-quality private optical networks and precise timing synchronization hardware.

1.2 Network Architecture and Challenges Based on Split Option 6

This thesis focuses on the architectural design and application of Split Option 6. As mentioned earlier, while adopting lower-layer splits (such as Split Option 7.2) can further centralize computational resources, it requires a more expensive PTP deployment environment and a transmission network with extremely high demands for low latency, which will lead to high capital and operational expenditures. In light of this, choosing the Split Option 6 approach, which leaves the complete physical layer (Monolithic PHY) at the remote node, can strike the best balance between resource requirements and cost-effectiveness. According to research by Khan et al. [9], cost-benefit trade-off analysis for 5G NR small cells indicates that compared to strict lower-layer splits, the architecture separating the MAC and PHY layers can significantly reduce the required fronthaul bandwidth while ensuring goodput, thereby achieving the goal of lowering overall deployment costs. Therefore, in non-ideal or resource-constrained network environments, Split Option 6 becomes the ideal choice that balances performance and cost.

1.2.1 Introduction to the FAPI Interface

Within the disaggregated RAN architecture, the Functional Application Platform Interface (FAPI) is standardized by the Small Cell Forum (SCF) for Split Option 6 (the MAC-PHY split) [10]. The FAPI design assumes that the MAC layer and the PHY layer are deployed on the same physical machine. Under this same-machine deployment, communication is performed using shared memory, which provides very low latency. The timing is driven by the PHY layer, which sends a `slot.indication` message to the MAC layer at every slot boundary. This message acts as a trigger to run the MAC scheduling loop. However, since FAPI relies on shared memory on a single host, it does not support physical separation or remote deployment of the base station components.

1.2.2 Introduction to the nFAPI Interface

To support remote deployment and physical separation under Split Option 6, the SCF defined the network Functional Application Platform Interface (nFAPI) [11]. The core design of nFAPI is to wrap FAPI messages into IP network packets and transmit them over packet-switched networks using network sockets [12]. This socket-based communication allows the virtualized MAC layer (operating as a Virtual Network Function or VNF) and the physical PHY layer (operating as a Physical Network Function or PNF) to be deployed on separate machines, providing high flexibility.

Unlike FAPI where the `slot.indication` directly triggers the MAC on the same machine, transmitting nFAPI messages over network sockets introduces transmission delays and jitter. To maintain synchronization, the SCF specification defines standard interface messages for node sync and delay management. Rather than fixing a single implementation, the specification provides generic inputs (such as observed packet arrival times and offsets) and generic outputs (such as timing adjustments). This generic input/output framework allows developers to build and use their own timing adjustment algorithms, preparing the system for flexible and customizable deployments in non-ideal networks.

It is particularly important to note that according to 3GPP TR 38.801 [2] specifications, the ideal fronthaul one-way latency limit for Split Option 6 is 250 μs. However, the SCF provides a more practical interpretation for nFAPI's latency tolerance in its specifications [11,13]. The SCF indicates that nFAPI is designed for packet-switched IP networks, and can support highly delayed and highly jittered transmission conditions, tolerating one-way latency ranging from 6 to 30 ms, by employing the built-in delay management mechanism of the nFAPI P7 interface and the SCTP protocol adapted for the P5 interface. This design specifically for non-ideal networks makes Split Option 6 highly practical and flexible in deployment.

1.2.3 Comprehensive Comparison of Split Technologies

To more intuitively present the differences between different split options, Table 1.1 provides a comprehensive comparison of Split 2, Split Option 6, Split 7.2x, and Split 8 in terms of protocol boundaries, bandwidth requirements, and latency limits.

This thesis focuses on the deployment and optimization of nFAPI because it provides excellent network flexibility, allowing network operators to logically split the base station and deploy it on two independent machines, even connected via general packet-switched networks. However, this physical split also brings great challenges: in a non-deterministic network environment, how to maintain time synchronization between the two

Table 1.1: 5G RAN Split Deployment Comparison Table

Split Option	**Split 2**	**Split 6 (nFAPI)**	**Split 7.2x**	**Split 8**
Protocol Boundary	PDCP / RLC	MAC / PHY	Low-PHY / High-PHY	RF / PHY
Fronthaul Data Type	PDCP PDU	MAC PDU (Packetized)	Frequency-domain IQ symbols	Time-domain IQ samples
Standardized Interface	F1	SCF nFAPI	O-RAN WG4 eCPRI	CPRI / eCPRI
Bandwidth Driver	Actual user traffic	Actual user traffic	Bandwidth, spatial layers	System bandwidth, antennas
Latency Limit [2,13]	1.5 $\sim$ 10 ms	250 μs (3GPP) 6 $\sim$ 30 ms (SCF)	250 μs	250 μs
Optimal Deployment	Non-real-time services	Small cells, indoor private networks	Macro cells	C-RAN, dedicated fiber

nodes and conduct precise delay management has become the primary difficulty in ensuring stable system operation.

1.3 Research Motivation and Problem Definition

Although Split Option 6 has flexible timing adjustment capabilities and better tolerance for non-ideal fronthaul networks, it still faces severe challenges in practical deployment. As radio access networks move towards virtualization and cloudification, the hosted MAC layer functions are typically deployed as Virtual Network Functions (VNFs) on general-purpose processors. However, in this general-purpose computing environment, non-deterministic operating system scheduling jitter and processing delay uncertainty often impact timing-sensitive baseband processing [6,14].

Specifically, in existing open-source implementations including OpenAirInterface (OAI), they have not truly implemented a generic nFAPI interface according to the SCF specifications. Instead, they use an incorrect approach that leads to high latency. They forward the raw `slot.indication` directly over the network to drive the MAC scheduling, and they rely on a statically configured *slots ahead* parameter (representing the scheduling ahead time of the VNF in units of slots) to handle delay. This design makes the system highly sensitive to packet-switched network jitter and operating system delays. To avoid packet loss in high-jitter environments, developers must set a very large static slots ahead value, which significantly increases transmission latency and defeats the benefits of the Split Option 6 architecture. When network congestion or CPU spikes occur, control messages still miss their deadlines, causing connection failures.

To address these limitations, this thesis re-develops the nFAPI interface based on the OAI project to support the standard node sync and delay management mechanisms. Specifically, this thesis addresses the following problem: how to maintain nFAPI Split 6 scheduling stability over non-deterministic packet-switched transport without relying on hardware-level synchronization or statically configured worst-case ahead times. The key challenge is that the scheduler must choose a slots ahead value before observing the future network delay. A conservative value improves robustness but increases MAC HARQ RTT, while an aggressive value reduces latency but risks late packet arrivals and slot loss. To resolve this tradeoff, we propose a closed-loop delay management process that uses PNF timing info to estimate the arrival margin and adapt the slots ahead value online. By providing generic inputs and outputs rather than hardcoded slot forwarding, our framework enables dynamic timing control and flexible deployment, and we contribute this re-developed nFAPI implementation back to the upstream OAI repository to support reproducible Split 6 research.

Therefore, the core research question of this thesis is defined as follows: In a cloud environment with non-deterministic processing loads and network jitter, how to dynamically and accurately compensate for end-to-end latency in the nFAPI Split Option 6 architecture to ensure the stability of timing synchronization?

1.4 Related Works

In recent years, the rapid development of the Open Radio Access Network (O-RAN) architecture has prompted many studies to optimize the performance and network latency management of the Split Architecture. For instance, the X5G testbed proposed by Villa et al. [8] demonstrated an advanced 5G O-RAN deployment solution based on hardware acceleration. This research successfully achieved commercial-grade peak throughput by offloading the massive computations of the Physical Layer (PHY) to dedicated NVIDIA GPUs and Mellanox SmartNICs. To guide researchers and operators in transitioning these complex virtualized architectures from software simulators to emulated and real-world testbeds, deployment solutions and software interoperability issues on open-source platforms like OpenAirInterface have been comprehensively surveyed [15].

To provide a highly flexible and cost-effective deployment solution, this thesis proposes a universal software-defined approach for non-ideal fronthaul environments. Unlike hardware-accelerated architectures like X5G, and unlike traditional methods that rely on static configurations to compromise on delay jitter, this thesis designs an Adaptive Slots Ahead Control Mechanism based on Exponentially Weighted Moving Average (EWMA) delay management specifically for the nFAPI architecture. This mechanism can run on general COTS servers, dynamically tracking and filtering network jitter, and actively adapting to unpredictable network congestion. Experimental results show that the method proposed in this thesis can effectively stabilize the HARQ loop latency (RTT) and maintain system throughput without relying on specialized hardware acceleration or precise clock synchronization equipment, providing a more cost-effective and flexible alternative for O-RAN deployment.

Simultaneously, current academic research on O-RAN delay management has explored various dimensions to address non-deterministic latency. At the intra-node level, Lee et al. [14] proposed an adaptive transmission strategy based on the FAPI interface, utilizing the variance in Layer 1 software (L1SW) processing times to opportunistically advance transmissions, which effectively complements end-to-end transport delay management. To further mitigate intra-node scheduling jitter on general-purpose servers, Aquifer [16] introduced transparent microsecond-scale operating system scheduling by exploiting common producer-consumer patterns in virtualized base stations. For strict deterministic latency requirements in ultra-reliable low-latency communication (uRLLC), Adamuz-Hinojosa et al. [17] introduced the ORANUS framework, employing Stochastic Network Calculus (SNC) to provide strict mathematical bounds for latency violations. Furthermore, from the perspective of User Equipment (UE) Service Quality (QoS), Lv et al. [18] developed UQ-vRAN, which emphasizes the joint optimization of MAC scheduling and higher-layer orchestration through precise traffic digital twins. Additionally, resource allocation for Hybrid Automatic Repeat Request (HARQ) retransmissions in mobile networks can be dynamically pre-allocated based on user equipment error rates and scheduling cycles, reducing transport block loss and stabilizing scheduler goodput [19]. From a computational perspective, AegisRAN [20] dynamically scales computing resources to achieve energy savings in virtualized base stations; however, deactivating unused CPU cores can introduce transient processing delay fluctuations, which further increases the timing synchronization challenges at the MAC-PHY interface.

To understand the delay bounds and capacity constraints of packet-switched transport networks, researchers have modeled fronthaul latency using queuing theory. Diez et al. [21] developed a comprehensive end-to-end queuing model based on Open Jackson Networks to analyze how background traffic and heterogeneous architectures influence latency across different split points, demonstrating that delay increases significantly as load approaches link capacity. This queuing-based framework was subsequently extended by Khan et al. [22] to model time-window relationships and priority-based scheduling over spine-leaf fronthaul topologies, providing operators with analytical tools to evaluate delay violations without relying on full network emulation.

While these existing literatures [14,16–23] have proposed optimization and delay-modeling schemes ranging from specific intra-node CPU adjustments to long-term QoS orchestration, they generally do not directly address the severe cross-node synchronization failures caused by non-deterministic fronthaul jitter in intermediate-layer splits. This thesis fills the critical gap by targeting the microsecond-level phase synchronization and latency challenges inherent in the nFAPI Split Option 6 architecture. Specifically, we focus on the fact that the Small Cell Forum (SCF) 225 nFAPI specification [11] defines the standard `Timing Info` message and interface-level delay management fields, yet existing open-source implementations only support static parameter initialization. Our proposed dynamic timing control completes this standardized framework by implementing an adaptive timing closed-loop feedback, presenting a unique academic contribution to realizing dynamic time synchronization under a pure software-defined network architecture without relying on specialized hardware.

To clearly position the proposed work within the current state-of-the-art (SOTA) literature, Table 1.2 summarizes the comparison across key architectural and performance characteristics.

Table 1.2: Comparison of Proposed Work with State-of-the-Art Solutions

Metric / Feature	**X5G [8]**	**Lee *et al.* [14]**	**Proposed Work**
Split Option	Split 7.2	Split 6 (FAPI)	Split 6 (nFAPI)
Fronthaul Type	Dedicated Fiber / TSN	Shared Memory (Intra-node)	Packet-switched Ethernet
Hardware Dependency	GPU / SmartNIC / PTP	COTS (No Acceleration)	COTS (No Acceleration)
Control Resolution	Symbol-level	OS / Process-level	Slot / TTI-level
Adaptation Domain	Hardware Offload	Intra-node CPU Scheduling	Cross-node Jitter & Delay

1.5 Thesis Contributions

In response to the aforementioned issues, the main contributions of this thesis are as follows:

- **Multi-dimensional Latency Quantitative Analysis for nFAPI Split Option 6:** We systematically analyzed and quantified the non-deterministic factors affecting synchronization failure, including Linux kernel scheduling jitter, VNF processing load, and network queuing delays.
- **Adaptive Slots Ahead Mechanism:** We proposed a dynamic adjustment mechanism based on node sync and delay management. It

uses an EWMA model to smooth timing information to estimate real-time jitter trends, and dynamically adjusts the slots ahead (S_{ahead}) to ensure the punctuality of scheduling messages.

- **End-to-End Validation under Controlled and Multi-hop Transport:** We implemented the proposed mechanism on the OpenAirInterface platform and combined it with a commercial UE and O-RU for verification. Experimental results show that this mechanism can significantly reduce missing slot packet errors and maintain stable network throughput under both Linux-TC-injected delay/jitter (under Experimental Scenario I) and a realistic 3-switch/3-router campus network deployment (under Experimental Scenario III).

- **Open-Source nFAPI Platform Contribution to Upstream OAI:** We submitted our adaptive timing control and nFAPI modifications as a merge request (MR) to the official OpenAirInterface (OAI) [24] repository. This contribution was merged into the official codebase. It provides a standard open-source nFAPI platform that follows the Small Cell Forum (SCF) specification for future development and testing.

Chapter 2 System Architecture

Before delving into system details, this study first introduces the adopted system framework. To integrate the network Functional Application Platform Interface (nFAPI) with mainstream commercial equipment, the system proposed in this study adopts a stable two-stage split architecture. The network is first connected through the Split 6 nFAPI interface between network functions, and then converted to the standard O-RAN Split 7.2x interface to connect to a commercial Pegatron O-RU. Figure 2.1 illustrates this system architecture and its delay management mechanism.

2.1 End-to-End Architecture

This system establishes a complete 5G end-to-end communication link extending from the core network to the user equipment:

- **Core Network (CN):** The core component responsible for routing, session management, and user data authentication.
- **O-RAN Central Unit (O-CU):** The logical node handling higher-layer protocol stacks, such as Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP). It connects to the distributed unit via the F1 interface. Together with the O-DU High, the O-CU belongs to the virtualized network functions (VNF) running on general-purpose virtual machines.
- **O-RAN Distributed Unit (O-DU):** To optimize computational per-

formance and deployment flexibility, the system divides the O-DU into O-DU High and O-DU Low. In this split, the O-DU High runs as a Virtual Network Function (VNF), whereas the O-DU Low and the underlying physical layer (Layer 1) are deployed on dedicated hardware as a Physical Network Function (PNF).

- **Switch:** A commercial network switch located between the VNF and PNF. This intermediate layer connection utilizes the nFAPI protocol. However, traversing this packet-switched network inevitably introduces non-deterministic latency.

- **O-RAN Radio Unit (O-RU):** A dedicated hardware unit responsible for Low PHY processing, Digital Beamforming, and Radio Frequency (RF) transmission.

- **Commercial Off-The-Shelf User Equipment (COTS UE):** Standard commercial smartphones or modems acting as the network's terminal receivers.

2.2 O-DU High: MAC, Scheduler, and Timing Core

Within the O-DU High virtualized network function, the timing control is organized as a unified timing control system. The core processes handling this timing core include the scheduler, node sync, and delay management:

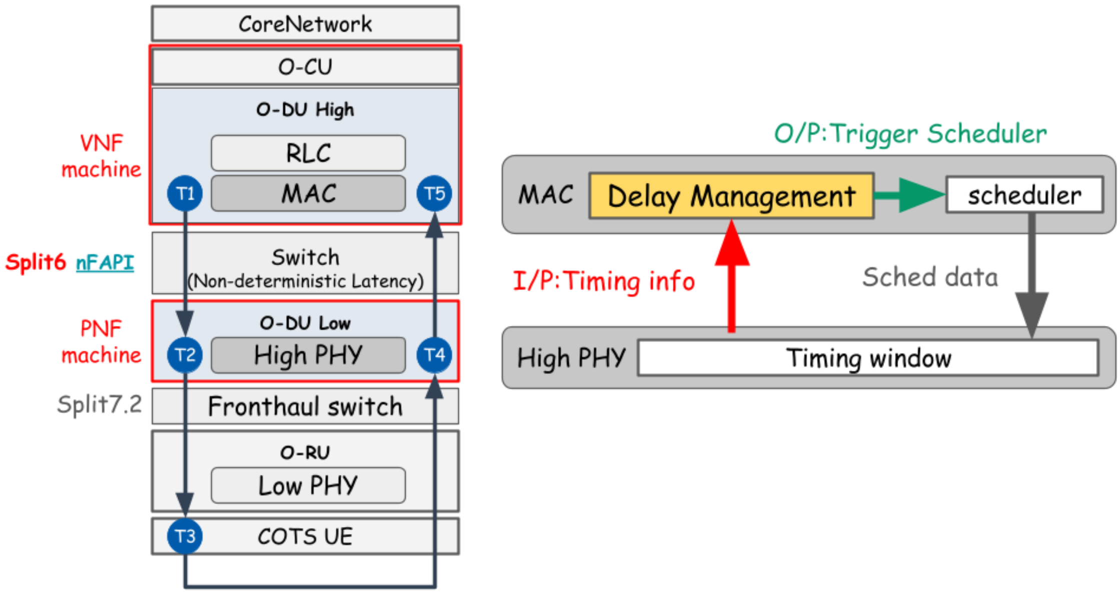


Figure 2.1: System architecture and delay management mechanism.

- **Scheduler:** Responsible for allocating available radio resources (e.g., time-frequency resource blocks) and determining the exact priority and timing of downlink/uplink data transmission.

- **Node sync:** Responsible for software-level clock synchronization. This procedure implements a latency probe procedure by periodically exchanging synchronization timestamps ($t_1 \sim t_4$) representing microsecond-level time-reference offsets between the VNF and PNF. It calculates the instantaneous clock offset and applies slot-level (S_{adj}) and microsecond-level (μs_{adj}) adjustments to align the VNF's local clock to the PNF baseline, eliminating long-term hardware clock drift (Clock Drift).

- **Delay Management:** Operates on top of the stabilized clock phase established by node sync. This mechanism utilizes the PNF Timing Info ($TimingInfo[i]$) feedback from the PNF to dynamically estimate transport network jitter. It then actively adjusts the slots ahead

(S_{ahead}) trigger offset of the scheduler. Here, *slots ahead* (or *VNF slots ahead*) refers to the number of slots by which the VNF triggers the scheduler in advance relative to the reference timing info. This control compensates for the non-deterministic packet transport delays and VNF processing delays without inducing synchronization loss.

2.3 O-DU Low: High PHY and Timing Window

O-DU Low manages the High PHY baseband processing. At this level, the most critical synchronization mechanism is the nFAPI PNF Timing Window:

- **Timing Window:** A predefined and strict time interval. Let T_{window} denote the duration of this PNF timing window, and T_{slot} denote the subframe slot duration. When scheduling messages sent by O-DU High traverse the network, they must accurately arrive and fall exactly within this window. Otherwise, the High PHY will reject the message, resulting in the inability to process data for that slot.

- **Timing Info (*TimingInfo*$[i]$):** The PNF continuously monitors the exact timing info of control messages relative to the transmission deadline. This information is denoted as *TimingInfo*$[i]$, representing the remaining time margin within the timing window. The PNF periodically feeds this information back to the MAC layer for delay management. This establishes a closed-loop feedback control system to compensate for unpredictable transmission delay variations between the VNF and PNF machines.

2.4 Key Validation Metrics

To evaluate the stability, synchronization accuracy, and real-time performance of the proposed algorithm in non-ideal environments, this study defines the following key validation metrics:

- **MAC Layer HARQ RTT ($T_5 - T_1$):** Specifically measures the Hybrid Automatic Repeat Request (HARQ) loop delay at the MAC layer. This metric precisely reflects the feedback efficiency and operational health of the link layer.
- **VNF-PNF Downlink Delay ($T_2 - T_1$):** Measures the accumulated one-way downlink latency from the virtualized machine (VNF) to the physical machine (PNF). This metric is used to directly quantify and analyze the delay jitter impact introduced by the intermediate Ethernet switch.

Note: RTT stands for Round-Trip Time. To verify the stability of the split architecture under non-ideal fronthaul conditions, this study simulates network delays using Linux Traffic Control (TC) on the uplink ($T_5 - T_4$) and downlink ($T_2 - T_1$) segments.

To validate the proposed performance optimization in a realistic network environment, the system design utilizes a commercial-grade deployment framework:

- **O-RAN Split 7.2x Interoperability.**
  Although the Small Cell Forum (SCF) has standardized the **network Functional Application Platform Interface (nFAPI)** [12] for the MAC-PHY split (Option 6), commercial Radio Units (O-RUs) natively support O-RAN Split Option 7.2x. To achieve seamless integration with open-source protocol stacks like OpenAirInterface [25], this system bridges these interfaces to ensure full-scale commercial interoperability.

- **Hybrid System Architecture for End-to-End Verification.**
  To verify the performance of nFAPI in a fully commercial ecosystem, this paper adopts a **hybrid architecture** (Split Option 6 + Split Option 7.2). In this model, the system provides a Translation/Interworking Layer that enables the nFAPI-driven MAC/High-PHY to successfully interact with the commercial Pegatron O-RU. This setup allows the use of **Commercial Off-The-Shelf (COTS)** user equipment to verify nFAPI performance in a real-world end-to-end environment.

Chapter 3 Proposed Method

This chapter details the proposed dynamic timing adjustment framework. To address the unstable transmission latency during packet transmission between the PNF and VNF, we propose two core timing control algorithms: node sync for software-level clock synchronization and delay management for slot-level adaptive delay management. The timing control of this system is organized as a closed-loop system, ensuring that the VNF-side scheduling can dynamically track and compensate for both microsecond-level clock drift and slot-level transport latency variation. The two core algorithms cooperate as follows:

1. **Node sync:** Maintains phase and slot boundary alignment between the VNF and PNF. It periodically exchanges timestamps to calculate the absolute phase offset, correcting for hardware clock drift (Clock Drift).

2. **Delay Management:** Tracks and estimates packet transport delays and jitter at the slot level, dynamically adjusting the slots ahead (S_{ahead}) to guarantee that control messages fall within the PNF's strict timing window.

3.1 Node sync

To achieve software-level clock synchronization, the system implements a latency probe procedure compliant with the nFAPI specification.

In this procedure, the VNF periodically transmits a downlink node synchronization (DL Node Sync) message containing a parameter t_1, which represents the transmission time offset in microseconds relative to the VNF SFN/slot 0/0 time reference. Upon receiving this message at the transport layer, the PNF records t_2, representing the arrival time offset in microseconds relative to the PNF SFN/slot 0/0 time reference. The PNF then immediately responds by sending an uplink node synchronization (UL Node Sync) message back to the VNF, indicating the echoed t_1, the recorded arrival offset t_2, and a parameter t_3 representing the transmission offset of the uplink message relative to the PNF SFN/slot 0/0 time reference. When the VNF receives the UL Node Sync message, it records a local arrival timestamp t_4, representing the arrival time offset in microseconds relative to the VNF SFN/slot 0/0 time reference.

Because the nFAPI timestamps are constrained by a 1024 subframe number (SFN) loop, offsets wrap around every 10.24 seconds (10,240,000 μs). The algorithm calculates the differences $d_1 = t_2 - t_1$ and $d_2 = t_4 - t_3$, applying a wrap-around correction to align them within the interval $[-5,120,000, 5,120,000]$ μs. The VNF then calculates the instantaneous clock offset between their respective time references:

$$\textit{offset} = \frac{d_1 - d_2}{2} \tag{3.1}$$

To prevent timing adjustments from causing control loop oscillations, the synchronization control loop implements a state-based locking mechanism. The system has two synchronization states: locked ($\textit{sync\_locked} = 1$) and unlocked ($\textit{sync\_locked} = 0$). In the locked state, the VNF monitors the drift of the local clock. If the absolute offset exceeds the slot duration T_{slot}:

$$|\textit{offset}| > T_{\text{slot}} \tag{3.2}$$

the VNF unlocks the synchronization ($sync\_locked = 0$) to trigger re-synchronization. In the unlocked state, the VNF checks whether the offset has converged within the slot duration T_{slot}. If $|offset| \leq T_{\text{slot}}$, the system locks the synchronization state ($sync\_locked = 1$). Otherwise, the VNF decomposes the offset into a slot-level adjustment S_{adj} and a microsecond-level adjustment μs_{adj} based on the slot duration T_{slot}:

$$S_{\text{adj}} = offset/T_{\text{slot}} \tag{3.3}$$

$$\mu s_{\text{adj}} = offset \pmod{T_{\text{slot}}} \tag{3.4}$$

These adjustments are accumulated into the global adjustment variables:

$$slot\_adjustment \leftarrow slot\_adjustment + S_{\text{adj}} \tag{3.5}$$

$$us\_adjustment \leftarrow us\_adjustment - \mu s_{\text{adj}} \tag{3.6}$$

The negative sign in the microsecond adjustment reduces the local thread sleep time when the VNF clock is behind, speeding up the slot transition rate to catch up. The synchronization process is summarized in Algorithm 1.

Algorithm 1 Node sync

Input: t_1, t_2, t_3 (Timestamps from PNF *UL_node_sync*), t_4 (VNF local receive timestamp)

Output: *slot_adjustment* (Slot adjustment accumulator), *us_adjustment* (Microsecond adjustment accumulator)

Notation:

offset (Instantaneous timing offset), *sync_locked* (Synchronization lock state)

T_{slot} (Slot duration in μs), T_{wrap} (Wrap-around period, 10240000 μs)

WrapCorrection(x, T) (Corrects x within $[-T/2, T/2]$)

Phase I: Timing Offset Calculation

$d_1 \leftarrow \text{WrapCorrection}(t_2 - t_1, T_{\text{wrap}})$

$d_2 \leftarrow \text{WrapCorrection}(t_4 - t_3, T_{\text{wrap}})$

$\textit{offset} \leftarrow (d_1 - d_2)/2$

Phase II: Hysteresis Lock and Clock Adjustment

if $\textit{sync\_locked} = 1 \land |\textit{offset}| > T_{\text{slot}}$ **then**

    $\textit{sync\_locked} \leftarrow 0$ ▷ Unlock due to excessive clock drift

end if

if $\textit{sync\_locked} = 0$ **then**

    if $|\textit{offset}| \le T_{\text{slot}}$ **then**

        $\textit{sync\_locked} \leftarrow 1$ ▷ Lock synchronization

    else

        $S_{\text{adj}} \leftarrow \textit{offset}/T_{\text{slot}}$ ▷ Integer division

        $\mu s_{\text{adj}} \leftarrow \textit{offset} \pmod{T_{\text{slot}}}$

        $\textit{slot\_adjustment} \leftarrow \textit{slot\_adjustment} + S_{\text{adj}}$

        $\textit{us\_adjustment} \leftarrow \textit{us\_adjustment} - \mu s_{\text{adj}}$

    end if

end if

return *slot_adjustment*, *us_adjustment*

3.2 Delay Management

To counteract changing network latency and maintain scheduling stability, a dynamic delay management strategy is implemented. This strategy operates as a closed-loop feedback mechanism based on standard Small Cell Forum (SCF) PNF Timing Info reported by the PNF. By evaluating the Timing Info relative to the PNF deadline, the algorithm dynamically adjusts the scheduling ahead time S_{ahead}.

3.2.1 Timing Model

To formalize the delay management problem, this section introduces the mathematical timing model of the nFAPI-based Split Option 6 architecture. Let T_{slot} denote the slot duration and S_{ahead} denote the number of slots by which the VNF scheduler triggers earlier than the target slot. For a scheduling message generated at VNF time t_g, the corresponding target PNF processing deadline is determined by the target slot boundary. The Timing Info reported by the PNF is denoted by $\textit{TimingInfo}[i]$, where $\textit{TimingInfo}[i] < 0$ indicates that the message arrives before the deadline (early arrival), and $\textit{TimingInfo}[i] > 0$ indicates a late arrival. Therefore, the delay management objective is to select the minimum stable S_{ahead} such that late arrivals are avoided ($\textit{TimingInfo}[i] \leq 0$) while the MAC HARQ Round-Trip Time (RTT) is not unnecessarily inflated.

3.2.2 EWMA-based Timing Info Estimation

When functional splits are deployed over packet-switched networks, the jitter of packet latency varies a lot with the network load. To address this, the proposed method references the congestion avoidance and control algorithm proposed by Jacobson and Karels [26] to introduce an Exponentially Weighted Moving Average (EWMA) filter that tracks both the mean and the deviation of the timing info. This filter smooths sudden network jitter, and its weight parameters can be changed into bitwise shift operations to reduce the computing overhead in the RAN underlayer [26]. Specifically, an EWMA filter is used to estimate the mean late arrival $\mathit{TimingInfoEWMA}[i]$ and the arrival jitter deviation $\mathit{TimingInfoDev}[i]$ based on the observed PNF Timing Info $\mathit{TimingInfo}[i]$ at slot i. The mean estimation is updated as:

$$\begin{aligned} \mathit{TimingInfoEWMA}[i] = (1-\alpha)&\mathit{TimingInfoEWMA}[i-1] \\ &+ \alpha \mathit{TimingInfo}[i] \end{aligned} \tag{3.7}$$

where the smoothing factor is chosen as $\alpha = 1/8$ to track the long-term trend, and the initial value is set to the first observed Timing Info value.

The jitter deviation $\mathit{TimingInfoDev}[i]$ is updated to capture timing variation as:

$$\begin{aligned} \mathit{TimingInfoDev}[i] = (1-\beta)&\mathit{TimingInfoDev}[i-1] \\ &+ \beta|\mathit{TimingInfo}[i] - \mathit{TimingInfoEWMA}[i]| \end{aligned} \tag{3.8}$$

where $\beta = 1/4$ is the deviation smoothing factor, and its initial value is set to half of the absolute first observed Timing Info value. To design the decision thresholds for the slots ahead, we use the network latency estimation parameters defined in the IETF RFC 6298 standard [27]. Specifically, we set the mean gain to $\alpha = 1/8$ and the deviation gain to $\beta = 1/4$, and use four times the deviation as a safety margin [27]. This parameter combination is a standard that has been tested in the industry for a long time. It covers most extreme network delays in a safe and stable way, ensuring that the nFAPI Split 6 architecture keeps strict timing synchronization and system stability under unpredictable jitter. These parameters are chosen as negative powers of two ($\alpha = 2^{-3}$, $\beta = 2^{-2}$) to allow the algorithm to perform fast bitwise shift operations in real time.

3.2.3 Adaptive Slots Ahead Control

The slots ahead trigger timing S_{ahead} is dynamically updated using a fast-increase and conservative-decrease control law. The adjustment logic is executed upon receiving each Timing Info report.

The decision rules are defined as follows:

- **Rapid Increase (Preserve Throughput):** If the smoothed worst-case arrival estimate (representing timing risk $\mathit{TimingInfoEWMA}[i] + \mathit{TimingInfoDev}[i]$) crosses the deadline ($\mathit{TimingInfoEWMA}[i] + \mathit{TimingInfoDev}[i] > 0$), the slots ahead value is immediately increased to prevent late packet arrivals and slot dropouts:

$$S_{\text{next}} = S_{\text{curr}} + \left\lceil \frac{\mathit{TimingInfoEWMA}[i] + \mathit{TimingInfoDev}[i]}{T_{\text{slot}}} \right\rceil \tag{3.9}$$

- **Gradual Decrease (Reduce Latency):** If estimated latency shows a safe margin (worst-case arrival is early enough), i.e., when the sum $\mathit{TimingInfoEWMA}[i] + 4 \times \mathit{TimingInfoDev}[i] < -T_{\text{slot}}$, the trigger timing is decremented by one slot to reduce latency:

$$S_{\text{next}} = S_{\text{curr}} - 1 \tag{3.10}$$

- **Maintain Baseline:** Otherwise, the trigger timing remains the same:

$$S_{\text{next}} = S_{\text{curr}} \tag{3.11}$$

To ensure the trigger timing remains within valid operational boundaries, the adjusted value is clamped to the range $[1, \lfloor T_{\text{window}}/T_{\text{slot}} \rfloor]$:

$$S_{\text{next}} \leftarrow \max\left(1, \min\left(S_{\text{next}}, \left\lfloor \frac{T_{\text{window}}}{T_{\text{slot}}} \right\rfloor\right)\right) \tag{3.12}$$

where the trigger timing is bounded by 1 and the maximum number of slots in the timing window $\lfloor T_{\text{window}}/T_{\text{slot}} \rfloor$.

The core delay management strategy is condensed in Algorithm 2.

3.2.4 Coordination between Node Sync and Delay Management

To ensure stable operation under real-world conditions, node sync and delay management must coordinate without inducing control loop oscillations. The microsecond-level node sync focuses strictly on correcting clock phase offsets to align the VNF's scheduling thread with the PNF's slot transitions. Because it directly shifts the VNF's time base, any clock adjustment $(S_{adj}, \mu s_{adj})$ temporarily shifts the arrival timestamp reference. Delay management is designed to absorb this transient shifting. By relying on history-weighted EWMA-based risk tracking (*TimingInfoEWMA*$[i]$

Algorithm 2 Delay Management

Input: $\mathit{TimingInfo}[i]$ (Observed PNF Timing Info),

S_{curr} (Current trigger timing),

T_{window} (Timing window), T_{slot} (Slot duration)

Output: S_{next} (Trigger timing for the next MAC schedule)

Notation:

$\mathit{TimingInfoEWMA}[i]$, $\mathit{TimingInfoDev}[i]$:

smoothed mean and arrival jitter

α, β: smoothing factors

Phase I: Latency and Jitter Estimation

$\mathit{TimingInfoEWMA}[i] \leftarrow (1-\alpha)\mathit{TimingInfoEWMA}[i-1]$

$+\alpha \mathit{TimingInfo}[i]$

$\mathit{TimingInfoDev}[i] \leftarrow (1-\beta)\mathit{TimingInfoDev}[i-1] +$

$\beta|\mathit{TimingInfo}[i] - \mathit{TimingInfoEWMA}[i]|$

Phase II: Adjustment Decision

if $\mathit{TimingInfoEWMA}[i] + \mathit{TimingInfoDev}[i] > 0$ **then**

$S_{\text{next}} \leftarrow S_{\text{curr}} +$

$\left\lceil \frac{\mathit{TimingInfoEWMA}[i]+\mathit{TimingInfoDev}[i]}{T_{\text{slot}}} \right\rceil$ ▷ Rapid Increase

else if $\mathit{TimingInfoEWMA}[i] + 4 \times \mathit{TimingInfoDev}[i] < -T_{\text{slot}}$ **then**

$S_{\text{next}} \leftarrow S_{\text{curr}} - 1$ ▷ Gradual Decrease

else

$S_{\text{next}} \leftarrow S_{\text{curr}}$ ▷ Maintain Baseline

end if

$S_{\text{next}} \leftarrow \max\left(1, \min\left(S_{\text{next}}, \left\lfloor \frac{T_{\text{window}}}{T_{\text{slot}}} \right\rfloor\right)\right)$ ▷ Clamp boundaries

return S_{next}

$+$ *TimingInfoDev*$[i]$), delay management avoids reacting to the temporary timing offsets caused by clock adjustments, allowing node sync to converge. Once node sync locks synchronization (*sync_locked* $= 1$), the time base stabilizes, allowing delay management to achieve precise slot-level optimizations (S_{ahead}).

Chapter 4 Experimental Results

This study conducts experiments in an OpenAirInterface (OAI) nFAPI end-to-end environment to verify the proposed delay management method. To ensure the statistical reliability of the evaluation, each throughput value reported represents the average of 180 data points (3 independent trials $\times$ 60 samples per trial). Unless specified otherwise, the default nFAPI parameters used across all evaluations are configured with a timing window of 6000 μs, a periodic timing info mode, and a timing info period of 1 slot.

To evaluate the performance of our proposed closed-loop synchronization and delay management algorithms, the experiments are designed to verify several key aspects of the system. First, we examine whether the EWMA-based filtering effectively stabilizes the noisy PNF timing info feedback under transport network jitter. Second, we evaluate whether the adaptive slots ahead method can maintain stable throughput while minimizing unnecessary MAC HARQ RTT compared with static baselines. Third, we test the system performance under a dynamic offered-load sweep without softmodem resets. Fourth, we validate the stability of the closed-loop delay management under severe network congestion and dynamic jitter. Fifth, we analyze the sensitivity of the EWMA parameters α and β to evaluate system stability and deadline-miss rates. Sixth, we verify whether the node sync algorithm dynamically corrects the time-base offset and maintains alignment under a multi-hop campus network. Seventh, we evaluate the ping latency under different throughput and real deployment scenarios. Finally, we study the impact of different timing info reporting modes and periods on the packet loss rate under different transport distances.

The rest of this chapter is organized as follows: Section 4.1 outlines the hardware, software, and the automated rApp evaluation framework, defining Experimental Scenario I, Experimental Scenario II, and Experimental Scenario III. Section 4.2 presents the validation of the EWMA filter under Experiment 1: Validation of EWMA for PNF Timing Info. Section 4.3 evaluates the throughput and latency performance under Experiment 2: Throughput & MAC RTT efficiency against static baselines. Section 4.4 analyzes the system behavior under Experiment 3: Dynamic Offered Load Sweep without Reset. Section 4.5 validates the closed-loop delay management stability under Experiment 4: Stability under Traffic Control Network Congestion. Section 4.6 investigates parameter configurations under Experiment 5: Sensitivity Analysis of EWMA Parameters α and β. Section 4.7 evaluates the node sync accuracy under Experiment 6: Synchronization Offset. Section 4.8 assesses the ping latency under Experiment 7: Ping Latency under Different Throughput. Finally, Section 4.9 analyzes the impact of reporting modes and periods under Experiment 8: Timing Info Mode.

4.1 Experimental Scenario

To evaluate the performance of our proposed timing adjustment framework under different environments, we design three topologies:

- **Experimental Scenario I (Controlled Single-Switch environment):** A standard testbed where the VNF and PNF are connected directly via a single commercial Ethernet switch, as shown in the rApp automated evaluation framework (Figure 4.1). This controlled setup is used to analyze EWMA filtering, benchmark throughput-latency trade-offs against static baselines, and evaluate robustness under delay and jitter. Specifically, Linux Traffic Control (TC) is applied between the nodes to inject artificial latency and jitter: on the downlink fronthaul link ($T_2 - T_1$) and on the uplink MAC feedback link ($T_5 - T_4$).

- **Experimental Scenario II (Near-Site Single-Hop Deployment):** A near-site campus deployment where the VNF and PNF are co-located in the same campus area and connected via a single commercial Ethernet switch, introducing minimal base transport latency (around 0.3–0.5 ms). This scenario, together with Scenario III, is deployed in the campus network topology illustrated in Figure 4.2. It evaluates the proposed timing control under stable, low-latency fronthaul conditions.

- **Experimental Scenario III (Multi-Switch/Router Campus Deployment):** A realistic multi-hop campus network topology where the VNF and PNF are physically disaggregated across different campus buildings. As shown in Figure 4.2, the connection traverses 3 commercial switches and 3 commercial routers, introducing realistic transport delays and route-dependent jitter (ping latency up to 2.0 ms).

The detailed hardware and software configuration of the end-to-end network deployment in the testbed environment is shown in the table below. Running the VNF and PNF on identical DGX Spark units ensures symmetric hardware processing capabilities (identical CPU architecture/frequency, cache sizes, memory speed, and NIC drivers).

Table 4.1: Testbed Node Configuration

Unit	**Specification**
Core Network (CN)	Open5GS [28]
Software Protocol Stack / Branch	OpenAirInterface [29] (`2026.w13`)
VNF Server	Intel® Xeon® Gold 6226R [30] with NetXtreme BCM5719 NIC [31,32]
PNF Server	Intel® Xeon® Gold 6433N [33] with Intel I210 NIC [32,34]
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
Commercial Off-The-Shelf (COTS) UE	Samsung Galaxy S20 [35]

Table 4.2: Wireless System Parameters

Parameter	**Setting**
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul Interface	O-RAN F release

To ensure consistent data quality for automated measurement and analysis, this study developed an automated evaluation rApp running on the

Non-Real-Time RAN Intelligent Controller (Non-RT RIC) within the O-RAN Service Management and Orchestration (SMO) framework [36,37], as shown in Figure 4.1. This setup serves as the testbed architecture for Experimental Scenario I. This rApp automatically starts the VNF and PNF. Once the gNB is successfully established, the rApp automatically connects to the control PC via SSH, toggles the UE's airplane mode using the Android Debug Bridge (ADB) [38], and then launches an iPerf [39] server on the Core Network server via SSH. Subsequently, it automatically executes a series of predefined iPerf test cases. After that, the rApp collects all log files via O1 SFTP and uses Python scripts to automatically plot charts for subsequent data analysis. During the experiments, we also used tcpdump [40] for packet sniffing, and utilized Linux PTP [41] and TSN time synchronization mechanisms [42] to ensure baseline timing among network nodes, while underlying data transmission relied on DPDK [43] for packet acceleration. A key capability of this framework is the ability to automate network delay simulations by running Linux Traffic Control (TC) on the intermediate link. Specifically, Linux TC is applied to inject latency and jitter on the fronthaul link ($T_2 - T_1$) and on the MAC feedback link ($T_5 - T_4$), which is crucial for verifying the stability of the disaggregated architecture under non-deterministic latency.

4.1.1 Thorough System-Level Integration and Validation

To prove the stability, practical deployability, and compatibility of the proposed timing control and nFAPI modifications, our software implementation was submitted and successfully merged into the official upstream

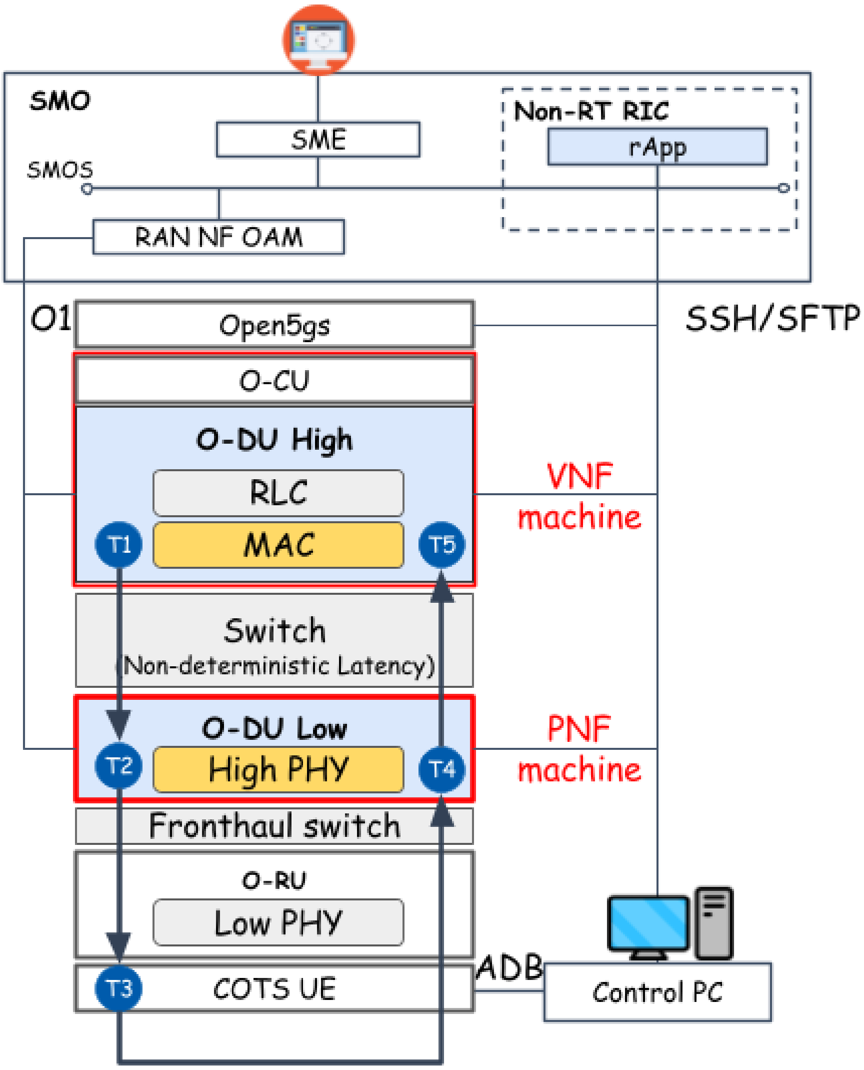


Figure 4.1: Architecture of the rApp automated evaluation framework for Experimental Scenario I (Controlled Single-Switch setup with Linux TC delay injection).

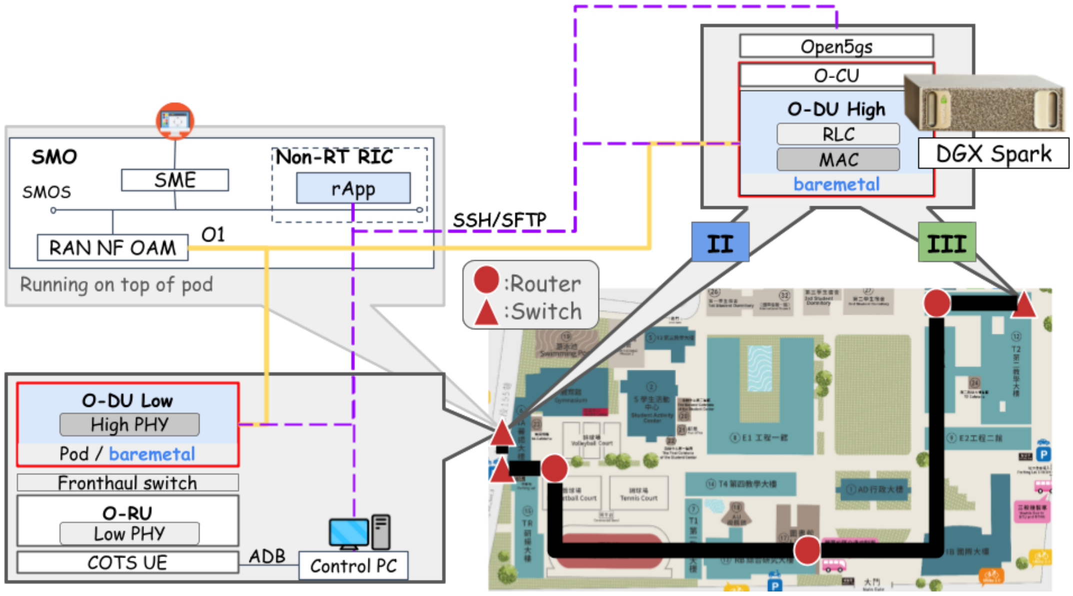


Figure 4.2: Experimental Scenario II and III: campus network deployments (near-site single-hop and multi-switch/router multi-hop) for evaluating the proposed adaptive timing control under realistic transport.

OpenAirInterface (OAI) repository [44]. Beyond our localized end-to-end evaluation using the rApp framework, the proposed solution went through thorough testing in the official OAI Continuous Integration and Continuous Deployment (CI/CD) pipeline [24]. This automated pipeline executes multi-platform builds and physical layer regression suites to ensure production-grade software quality.

In the software compilation and image deployment stage, the proposed implementation demonstrates high portability across heterogeneous hardware architectures. As compiled in Table 4.3, the cross-platform builds successfully passed on standard x86 servers, ARM64-based edge servers, and NVIDIA Jetson edge AI platforms under both Ubuntu and enterprise-grade Red Hat Enterprise Linux (RHEL) [45] environments. These results confirm that our proposed timing control is independent of specific hardware acceleration cards, allowing seamless deployment on general-purpose Commercial Off-The-Shelf (COTS) cloud infrastructure.

Table 4.3: OAI CI/CD Cross-Platform Compilation and Image Building Results

Stage	**Job / Builder**	**Target and Hardware Description**	**Result**
x86 Build	Ubuntu-Image-Builder	Build OAI network function images on standard x86 servers	Passed
ARM Build	Ubuntu-ARM-Image-Builder	Build images on ARM64 for ARM-based edge servers	Passed
Edge AI Build	Ubuntu-Jetson-Image-Builder	Build light-weight images for NVIDIA Jetson platforms	Passed
RHEL Build	RHEL-Cluster-Image-Builder	Build images for Red Hat Enterprise Linux (RHEL) clusters	Passed
Cross-Compile	ARM-Cross-Compile	Verify cross-compilation from x86 to ARM64 architecture	Passed

Following cross-platform compilation, our implementation was subjected to system integration and physical-layer dynamic testing. Table 4.4 summarizes the key integration and simulation tests that our code passed. The validation demonstrates that the slot-level scheduler adjustments (S_{ahead}) and microsecond-level clock corrections ($S_{adj}, \mu s_{adj}$) do not break 3GPP-compliant physical layer procedures [46] or F1/N2 standard interface control-plane handovers [47]. Crucially, the code successfully integrated with commercial Quectel modules under standard RF conditions, proving that the proposed software-defined timing loops successfully satisfy the strict timing window constraints of real-world physical basebands.

These regression and compilation results prove that the proposed dy-

Table 4.4: Summary of System Integration and Physical Layer Validation

Validation Scope	Passed Job Name	Description and Highlights
E2E 5G SA and RIC Integration	OAI-FLEXRIC-RAN-Integration	Run in 5G SA mode, integrate with 5G Core and FlexRIC (near-RT RIC) [48] with low packet loss. This validation uses the E2 interface protocol [49].
Commercial UE and RF Integration	RAN-NSA-B200-Module-LTEBOX RAN-SA-B200-Module-SABOX	Run with USRP B200 and commercial Quectel modules to meet strict timing in real RF environments.
Third-Party RF Frontend Compatibility	RAN-SA-AW2S-CN5G	Run end-to-end tests with third-party AW2S RF equipment (20MHz TDD SA) [50] and Amarisoft UE.
Handover Stability	RAN-SA-Handover-CN5G	Pass F1-interface and N2-interface handover tests [47] under 5G SA to keep control plane stable.
5G L1 and Channel Compatibility	RAN-Channel-Simulation RAN-PhySim-GraceHopper-5G	Pass physical layer simulations (including LDPC, Polar, PBCH, MIMO, MCS) to prove timing changes do not break 3GPP rules [46]. Testing was also performed on GPU-accelerated platforms [51].

namic delay-aware timing control is robust, backward-compatible, and fully standardized, serving as a stable baseline for future O-RAN Option 6 deployments in the global open-source community.

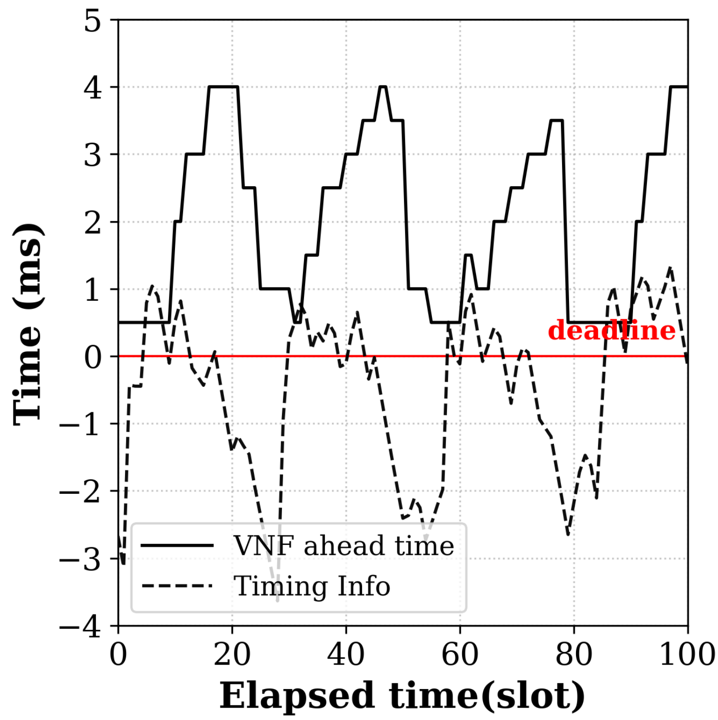
4.2 Experiment 1: Validation of EWMA for PNF Timing Info

This scenario verifies the effectiveness of applying the Exponentially Weighted Moving Average (EWMA) to smooth the PNF Timing Info ($\mathit{TimingInfo}[i]$). Due to network jitter and server processing delays, directly using the raw PNF Timing Info can lead to unstable scheduling decisions.

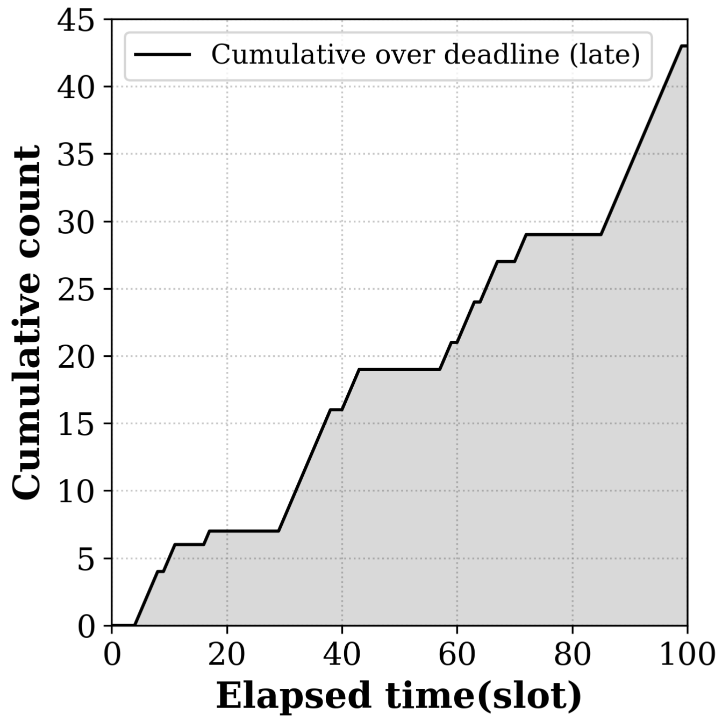
Before describing the results, we introduce the axes of the charts. For the timing tracking charts, Figure 4.3a and Figure 4.3c, the X-axis represents the elapsed slots and the Y-axis represents the PNF Timing Info value (in microseconds). For the deadline-miss cumulative charts, Figure 4.3b and Figure 4.3d, the X-axis represents the elapsed slots and the Y-axis represents the cumulative count of packets exceeding the deadline.

Figure 4.3a shows the timing control directly using the raw PNF Timing Info ($\mathit{TimingInfo}[i]$), where the raw timing values change a lot. Figure 4.3b represents the cumulative count of the packets that exceed the deadline when the timing adjustment directly uses the raw feedback. The EWMA mechanism filters out the high-frequency jitter, as shown in the PNF Timing Info ($\mathit{TimingInfo}[i]$) after applying EWMA in Figure 4.3c. Finally, Figure 4.3d shows the cumulative count of the packets exceeding the deadline under the EWMA-based timing control.

The main conclusion is that applying EWMA filters out high-frequency transport network jitter, allowing the timing loop to adjust the slots ahead stably and significantly reducing the count of late-arriving packets to prevent packet loss.



(a) Direct Use of *TimingInfo*$[i]$



(b) Cumulative Count of Packets Exceeding Deadline under Direct Use

Figure 4.3: EWMA processing validation for PNF Timing Info (*TimingInfo*$[i]$).

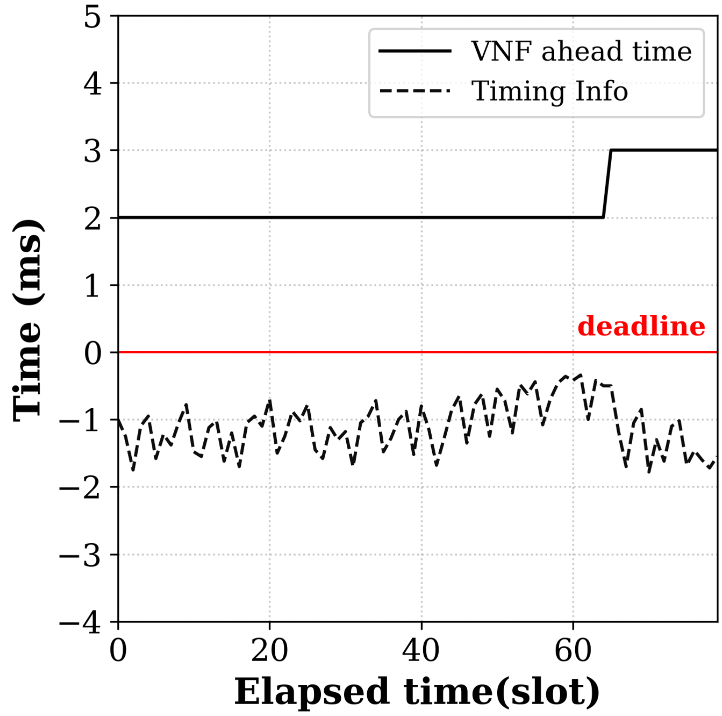
4.3 Experiment 2: Throughput & MAC RTT efficiency against static baselines

In this scenario, we evaluate the benchmark performance of the proposed adaptive timing mechanism against standard static slots ahead configurations. Figure 4.5 presents the performance evaluation.

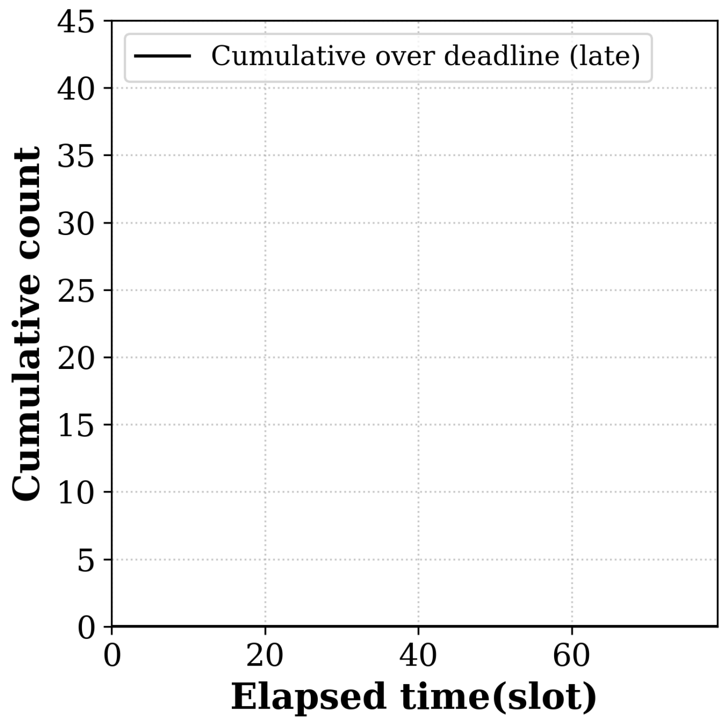
Before describing the results, we introduce the axes of the charts. In Figure 4.5a, the X-axis represents the offered load (in Mbps) and the Y-axis represents the achieved throughput (in Mbps). In Figure 4.5b, the X-axis represents the offered load (in Mbps) and the Y-axis represents the average MAC HARQ RTT (in milliseconds).

As shown in Figure 4.5a, when the offered load is low (between 100 Mbps and 800 Mbps), all configurations except the static 2 slots ahead configuration can track the offered load and sustain the target throughput. However, when the offered load increases to 1000 Mbps, static configurations with smaller slots ahead settings fail to maintain the throughput: the 4 slots ahead configuration levels off at about 830 Mbps, and the 6 slots ahead configuration levels off at about 870 Mbps. The 2 slots ahead configuration drops to near-zero throughput when the offered load goes beyond 200 Mbps because the fixed timing advance is too small. Although larger static configurations like 8 slots ahead can sustain the throughput at 940 Mbps under high load, they introduce larger RTT latency across all loads, as shown in Figure 4.5b.

The proposed method dynamically adjusts the slots ahead based on the feedback. For offered loads from 100 Mbps to 700 Mbps, it selects



(c) EWMA Application on $TimingInfo[i]$



(d) Cumulative Count of Packets Exceeding Deadline under EWMA

Figure 4.3: EWMA processing validation for PNF Timing Info ($TimingInfo[i]$) (cont.).

smaller slots ahead settings, maintaining a lower average MAC HARQ RTT of about 5.8 ms to 6.7 ms, which is lower than that of the static 4 slots ahead configuration (about 7.0 ms). When the offered load increases to 1000 Mbps, the proposed method dynamically increases the scheduling advance to sustain the target throughput. This increase causes the average MAC HARQ RTT to rise to about 7.9 ms at 1000 Mbps. However, this RTT is still lower than that of static 6 slots ahead (about 8.4 ms), 8 slots ahead (about 9.2 ms), and 10 slots ahead (about 10.6 ms) configurations.

The main conclusion is that the proposed method effectively minimizes MAC RTT latency while sustaining the target throughput under different offered loads, achieving an optimal latency-throughput trade-off.

4.4 Experiment 3: Dynamic Offered Load Sweep without Reset

To evaluate the system behavior under dynamic traffic changes, we conduct a continuous offered-load sweep using an iPerf3 client on the UE toward a CN-side server. The offered load is dynamically changed from 100 Mbps to 1000 Mbps in steps while the gNB-UE connection remains active, without any softmodem resets.

Figure 4.4 displays the results of this dynamic load sweep. For Figure 4.4a, the X-axis represents the elapsed time (in seconds) and the Y-axis represents the throughput (in Mbps). For Figure 4.4b, the X-axis represents the elapsed time (in seconds) and the Y-axis represents the dynamically adjusted slots ahead value.

As shown in Figure 4.4a, the system throughput tracks the offered load changes smoothly as the load steps up and down. Figure 4.4b demonstrates that the proposed method adjusts the slots ahead parameter on the fly to match the changing offered load. The slots ahead parameter increases under high load to prevent late packet arrivals and decreases under low load to minimize latency.

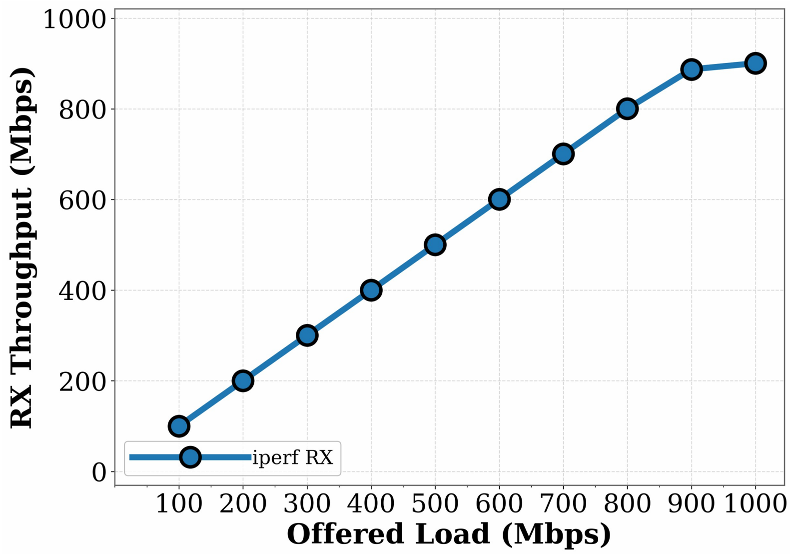
The main conclusion is that the proposed method adaptively adjusts the slots ahead parameter on the fly without requiring softmodem resets, successfully maintaining stable throughput and connection reliability under dynamically changing traffic loads.

4.5 Experiment 4: Stability under Traffic Control Network Congestion

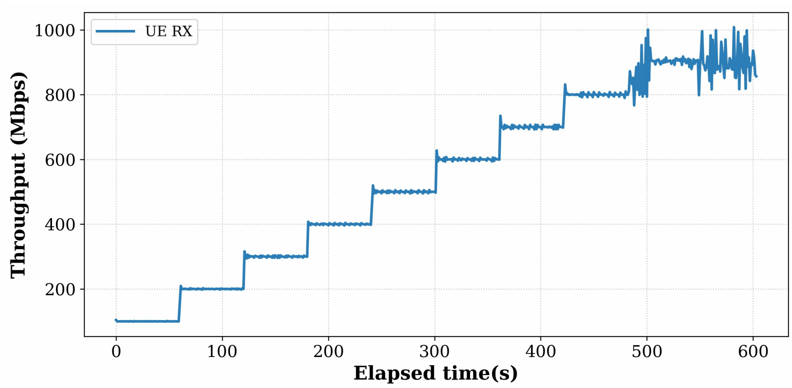
To test the system's stability, we utilized Linux TC to inject artificial network congestion into the $T_2 - T_1$ and $T_5 - T_4$ segments, altering the One-Way Delay (OWD) and jitter. Figure 4.6 displays the performance results under these conditions.

Before describing the results, we introduce the axes of the charts. In Figure 4.6a, the X-axis represents the injected latency (in milliseconds) and the Y-axis represents the achieved UE RX throughput (in Mbps). In Figure 4.6b, the X-axis represents the injected network jitter (in microseconds) and the Y-axis represents the achieved UE RX throughput (in Mbps).

As shown in Figure 4.6a, when the delay exceeds the fixed timing win-

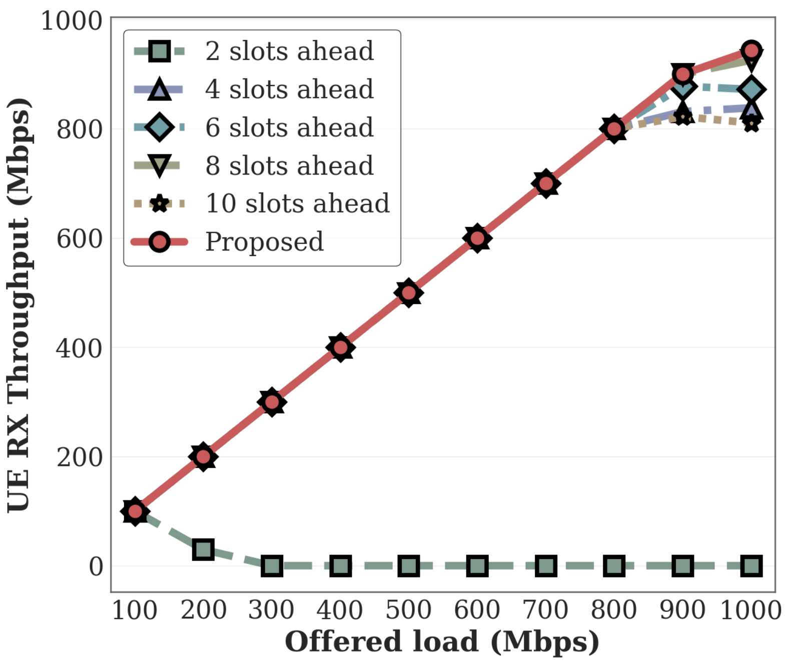


(a) Throughput Tracking

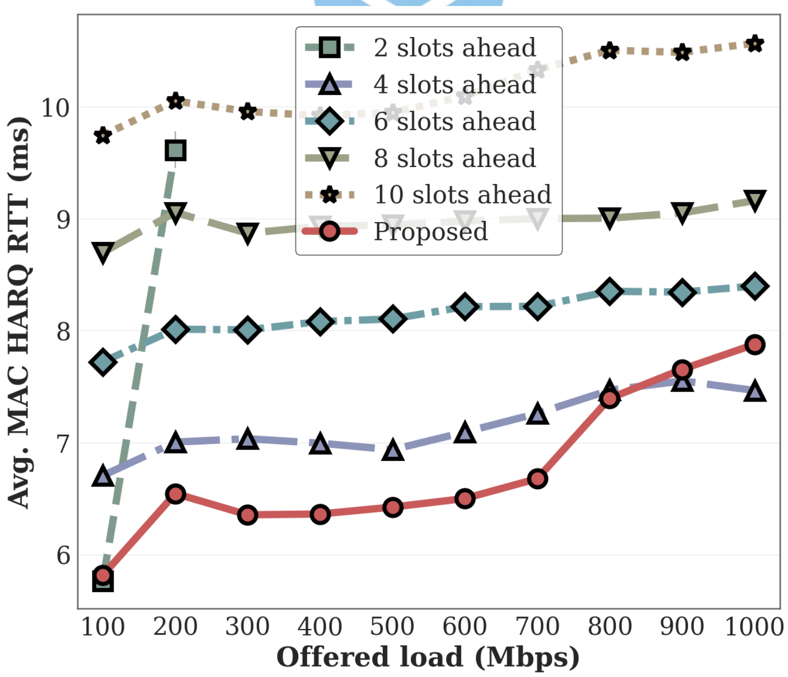


(b) Dynamic slots ahead Adjustment

Figure 4.4: System performance under dynamic offered-load sweep without softmodem resets.



(a) Throughput Optimization



(b) MAC Latency and Reliability

Figure 4.5: Performance Evaluation: MAC RTT Efficiency and System Throughput.

dow, static configurations experience a severe drop in throughput and frequent connection resets. In contrast, the proposed method dynamically expands the slots ahead (S_{ahead}). This expansion allows the MAC scheduler to trigger scheduling commands in advance, ensuring that packets can still arrive at the PNF within the valid timing window despite the increased transmission delay. Therefore, even at congestion levels that would cause complete synchronization failure under static configurations, the system can maintain stable throughput. Specifically, the proposed method maintains a throughput above 900 Mbps across all tested VNF-PNF latency values up to 3.0 ms, whereas the best-performing static configuration (10 slots ahead) degrades to near-zero throughput at approximately 2.5 ms injected latency.

To further evaluate the impact of unpredictable network jitter between the VNF and PNF, we perform a second experiment using Linux TC to simulate random jitter on the fronthaul link. In this test, we run iPerf UDP traffic with an offered load of 1 Gbps to fully saturate the downlink transmission. As shown in Figure 4.6b, when network jitter is high and unpredictable, fixed slots ahead settings cannot handle the timing variations, leading to packet losses, synchronization failures, and a major drop in throughput. On the other hand, the proposed method dynamically adjusts the scheduling advance, keeping the transmission timing aligned. As a result, the UE achieves and maintains stable and high DL RX throughput even under severe jitter conditions. Quantitatively, the proposed method exhibits graceful throughput degradation, maintaining over 900 Mbps at 0–600 μs jitter and approximately 750 Mbps even at 1000 μs jitter. In contrast, static configurations collapse sharply at their respective thresholds: the best-performing static configuration (6 slots ahead) drops below 300 Mbps at 800 μs and near zero at 1000 μs, while configurations with 4, 8, and 10 slots ahead all fail below 600 μs.

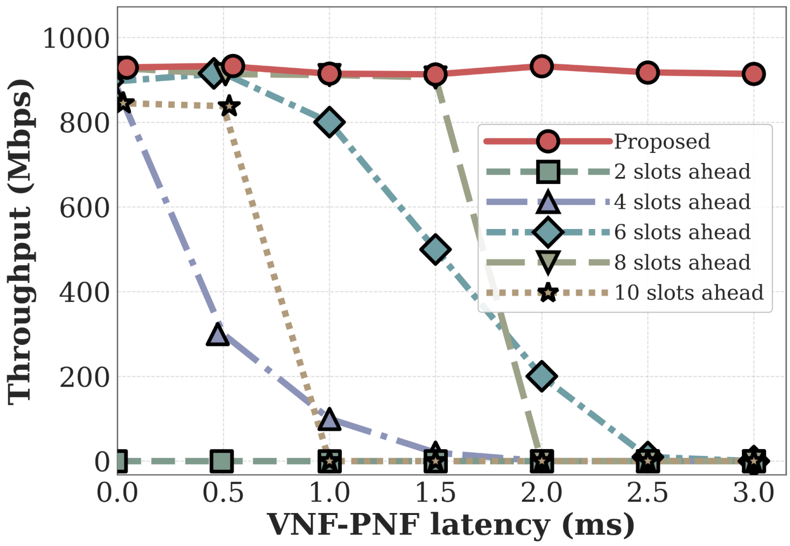
The main conclusion is that the proposed method ensures high stability and throughput under severe transport latency and jitter, whereas static configurations suffer from severe degradation or failure.

4.6 Experiment 5: Sensitivity Analysis of EWMA Parameters α and β

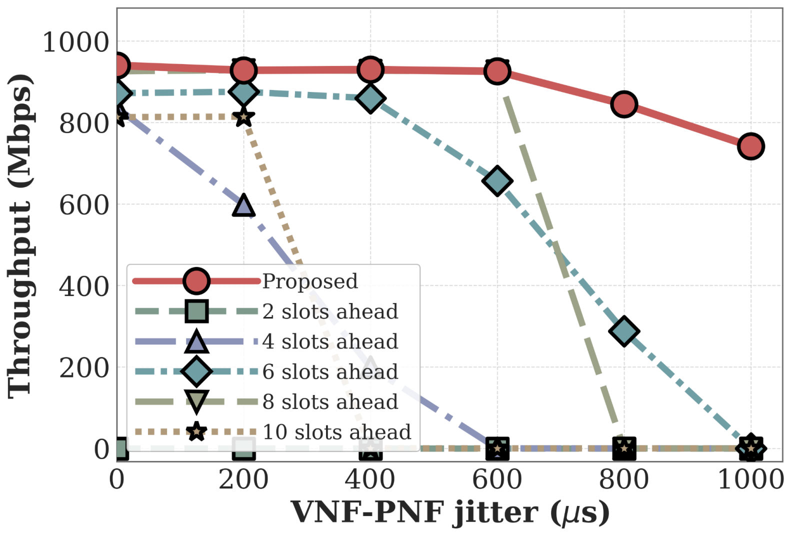
To fully evaluate the behavior of the proposed method under different EWMA filtering parameters, we design a sensitivity comparison experiment. This experiment cross-compares different parameter combinations by fixing α while changing β, and fixing β while changing α, to observe how parameter sensitivity affects system stability under network jitter.

Figure 4.7 presents the experimental results. Before describing the results, we introduce the axes of the charts. In the line chart of Figure 4.7a, the X-axis represents the elapsed slots and the Y-axis represents the slots ahead value. In the bar chart of Figure 4.7b, the X-axis represents the different parameter configurations, and the Y-axis represents the deadline-miss rate (in percentage).

From the line chart in Figure 4.7a, we observe that when the parameters are extreme, the system behavior degrades. For example, in the Danger-Corner configuration ($\alpha = 1/2$, $\beta = 1/32$) or the SluggishBeta config-



(a) System Stability under Network Congestion



(b) Throughput Performance under Unpredictable Jitter

Figure 4.6: System stability and throughput performance under network congestion and jitter.

uration, the timing adjustment loop reacts too strongly or converges too slowly to network jitter. This causes the slots ahead parameter to oscillate frequently, making it unable to maintain a safe buffer time. In contrast, the Default combination ($\alpha = 1/8$, $\beta = 1/4$) achieves excellent stability in tracking delay variations.

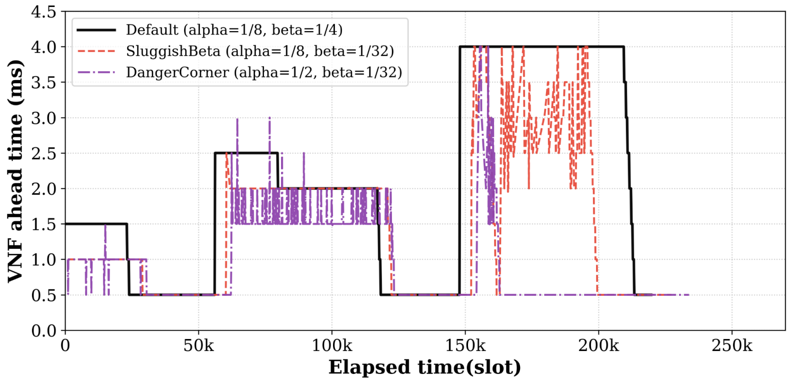
This stability is also reflected in the bar chart in Figure 4.7b. The Default configuration successfully reduces the deadline-miss rate to only 0.022%. This rate is about 32 times lower than that of the DangerCorner configuration, which has the most severe timing oscillations.

The main conclusion is that the Default parameter combination ($\alpha = 1/8$, $\beta = 1/4$) provides the best stability, effectively absorbing unexpected transport delays and ensuring high transmission reliability.

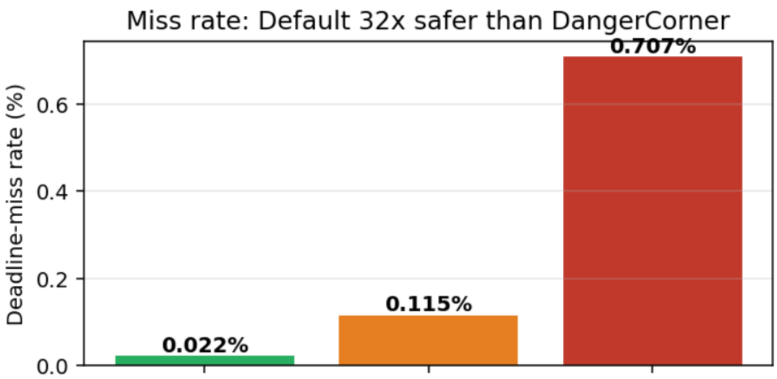
4.7 Experiment 6: Synchronization Offset

To evaluate the synchronization performance of the proposed node sync mechanism under a multi-hop transport network, we deploy the VNF and PNF across the campus network in Experimental Scenario III. As shown in Fig. 4.2, the VNF-side components are placed at a remote computing site, while the radio-side components and UE remain at the local radio site.

Figure 4.8 shows the measured VNF-PNF synchronization offset. Before describing the results, we introduce the axes of the chart: the X-axis represents the elapsed slots, and the Y-axis represents the synchronization offset (in microseconds).



(a) Dynamic slots ahead under Different Parameters



(b) Deadline-miss Rate Comparison

Figure 4.7: Sensitivity analysis of the EWMA parameters α and β.

When the connection between the VNF and PNF is established, they start calculating slot 0 independently. Program start-up timing differences cause an initial program execution timing offset. In addition, their clock phase drifts during operation. When the calculated offset accumulates and becomes larger than one slot (500 μs for subcarrier spacing of 30 kHz and slot duration of 500 μs), the system triggers the node sync algorithm to adjust the VNF's scheduling time base. This readjusts the VNF's slot boundaries to align with the PNF, correcting the time-base offset and stabilizing the synchronization offset within a small range of 0 to 100 μs.

The main conclusion is that the proposed node sync algorithm dynamically corrects the time-base offset and maintains high synchronization accuracy within a small range (0 to 100 μs) even under a realistic, non-deterministic campus network environment.

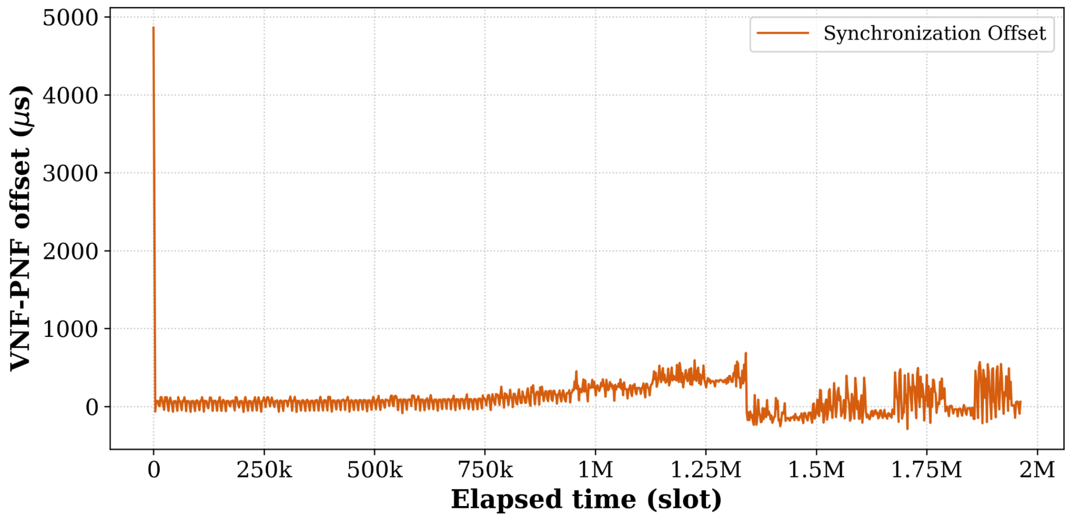


Figure 4.8: VNF-PNF synchronization offset over elapsed slots under the multi-switch/router deployment.

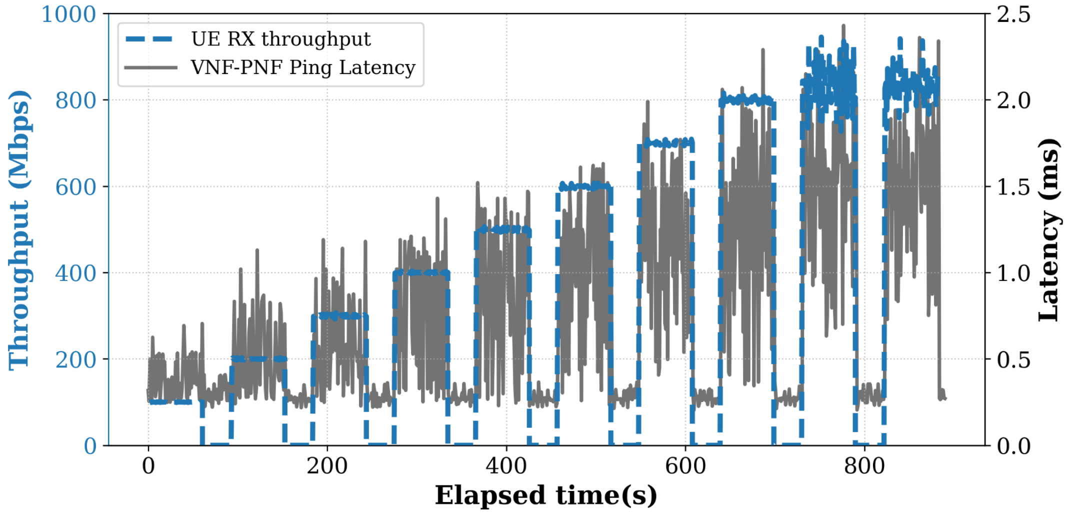
4.8 Experiment 7: Ping Latency under Different Throughput

To evaluate the system-level throughput stability of the proposed method under different real deployment conditions, we execute end-to-end data transmission under Experimental Scenario II and Experimental Scenario III. We use ping to measure the network interface latency between the VNF and PNF. Figure 4.9 shows the results.

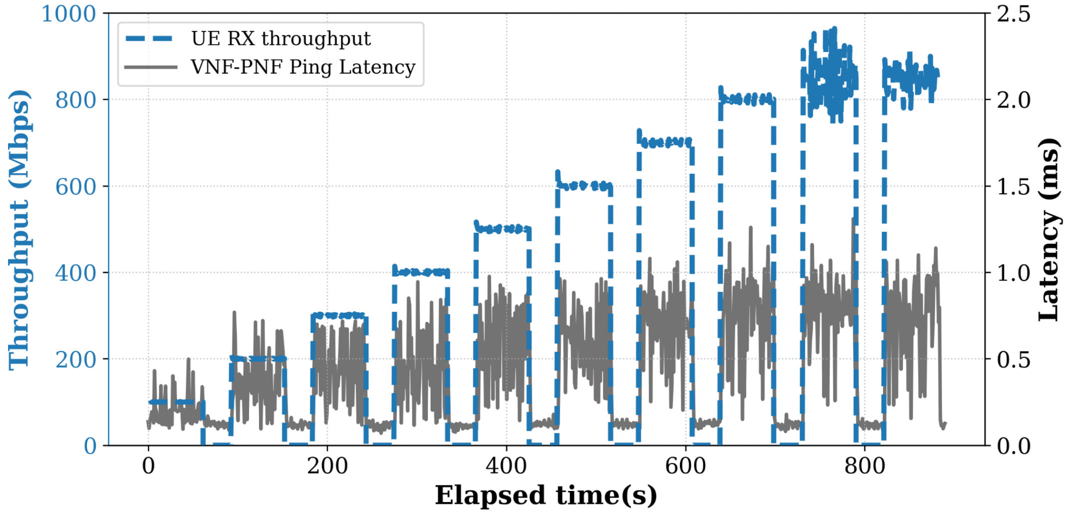
Before analyzing the results, we introduce the axes of the charts. In both Figure 4.9a (Scenario II) and Figure 4.9b (Scenario III), the X-axis represents the elapsed time (in seconds), and the Y-axis represents either the achieved UE RX throughput (in Mbps) or the ping latency (in milliseconds).

Under Scenario II (Near-site deployment, Figure 4.9a), the base latency is lower and more stable (around 0.3 to 0.5 ms). The proposed method maintains stable near-peak throughput. Because the transport delay is lower, the required slots ahead parameter is smaller, which also reduces the MAC HARQ RTT. Under Scenario III (Remote deployment, Figure 4.9b), the connection spans the campus network and traverses multiple switches and routers, resulting in higher base latency and queueing delay variations (ping latency up to 2.0 ms). Despite this severe transport jitter, the proposed method dynamically adapts the slots ahead parameter to prevent late packet arrivals.

The main conclusion is that the proposed method successfully maintains stable, near-peak throughput (close to 940 Mbps) in both near-site and remote multi-hop deployments, demonstrating high resilience to real-world transport latency and jitter.



(a) Scenario II: Near-site deployment



(b) Scenario III: Remote deployment

Figure 4.9: UE RX throughput and ping latency under different real network deployment scenarios.

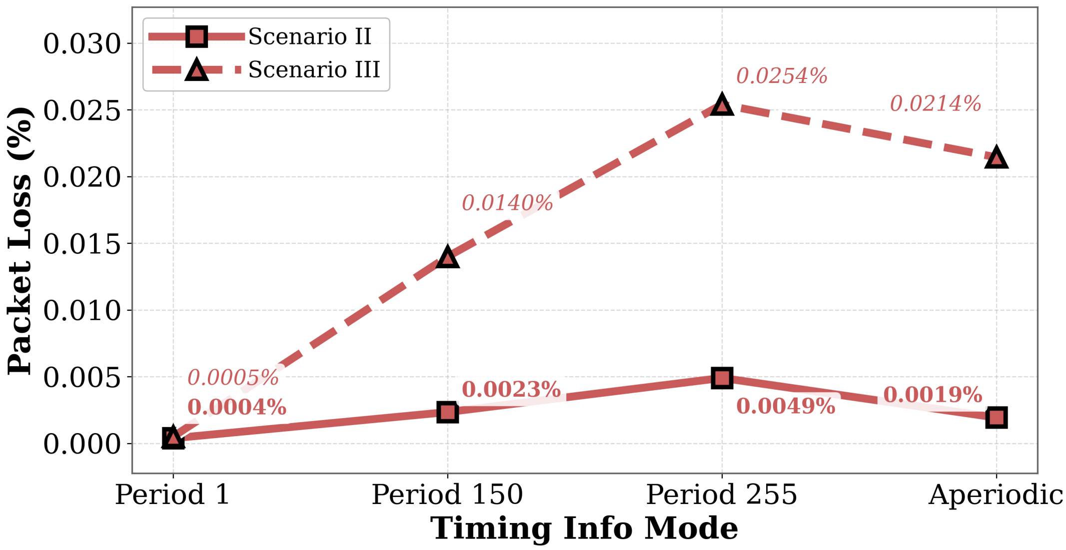
4.9 Experiment 8: Timing Info Mode

We evaluate the impact of different timing info reporting modes and periods under Scenario II (Near-site deployment) and Scenario III (Remote deployment). The nFAPI protocol supports periodic and aperiodic timing info feedback modes. Figure 4.10 presents the evaluation results.

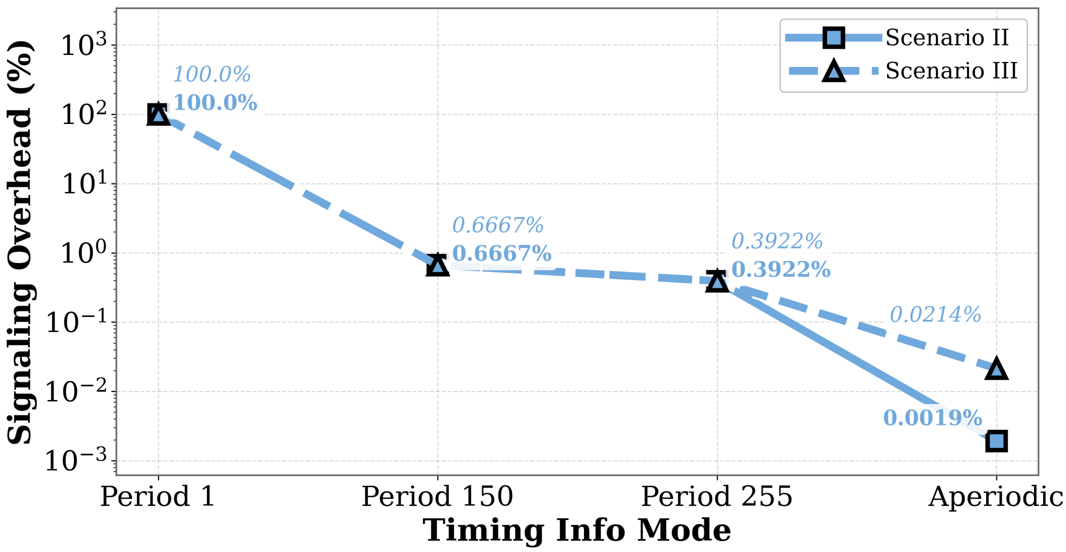
Before describing the results, we introduce the axes of the charts. In both Figure 4.10a (Scenario II) and Figure 4.10b (Scenario III), the X-axis represents the different reporting configurations (periodic periods in slots, and the aperiodic mode), and the Y-axis represents the packet loss rate (in percentage).

As shown in Figure 4.10, in both cases, increasing the timing period in periodic mode leads to a higher packet loss rate. This is because a longer period reduces feedback frequency. As a result, the VNF timing loop cannot adjust the slots ahead parameter fast enough to adapt to network variations, causing more slot packets to arrive late and be discarded by the PNF. Under Scenario III (Remote deployment), the longer distance and higher transport jitter result in a higher base packet loss rate, making the system more sensitive to feedback delay. Under Scenario II (Near-site deployment), the lower base latency and jitter allow the system to maintain a lower packet loss rate even with a slightly longer timing period.

The main conclusion is that using periodic mode with a short period (1 slot) or aperiodic mode is essential to keep the packet loss rate low (close to 0%) under transport networks with high jitter, whereas longer reporting periods degrade performance.



(a) Scenario II: Near-site deployment



(b) Scenario III: Remote deployment

Figure 4.10: Evaluation of packet loss rate under different deployment scenarios.

Chapter 5 Conclusion

This study addresses the transmission latency issues in the O-RAN Open Radio Access Network architecture by proposing an adaptive slots ahead mechanism, with a specific focus on the Option 6 functional split architecture utilizing the nFAPI interface. In a non-ideal packet-switched network environment, traditional static slots ahead configurations cannot effectively handle unpredictable network jitter and server processing delays, easily leading to scheduling timeouts and synchronization failures. To overcome this challenge, this study designs a dynamic delay management algorithm based on the Exponentially Weighted Moving Average (EWMA) model. Through closed-loop feedback control, it continuously tracks and estimates end-to-end network transmission latency characteristics.

By integrating the OpenAirInterface (OAI) open-source software with commercial Pegatron O-RU equipment into an end-to-end physical verification platform, the experimental results demonstrate that the proposed delay management architecture significantly improves system stability. Under simulated network congestion scenarios, this mechanism actively compensates for timing offsets, ensuring that MAC layer scheduling commands accurately fall within the timing window of the physical layer, thereby significantly reducing connection drops caused by missing slot packets within the evaluated range. Compared to traditional solutions that rely on expensive hardware acceleration or strict hardware-level PTP synchronization, the software-based solution proposed in this study demonstrates high deployment flexibility and cost-effectiveness on general-purpose Commercial Off-The-Shelf (COTS) servers, providing a reliable key technological foundation for the commercialization of future 5G/6G heterogeneous networks.

Although the proposed software-based timing control achieves robust performance, the current evaluation focuses primarily on downlink timing control under controlled delay and jitter injection. Future work will extend the proposed method to timing enhancements, multi-cell deployments, and analytical stability bounds for the feedback loop.

References

[1] O-RAN Alliance, "O-RAN Specifications." [Online]. Available: `https://www.o-ran.org/specifications`, 2025.

[2] 3rd Generation Partnership Project (3GPP), *TR 38.801 Study on new radio access technology: Radio access architecture and interfaces*, 2017. Version 14.0.0.

[3] B. Agarwal, R. Irmer, D. Lister, and G.-M. Muntean, "Open ran for 6g networks: Architecture, use cases and open issues," *IEEE Communications Surveys & Tutorials*, vol. 28, no. 3, pp. 2881–2924, 2025.

[4] L. M. P. Larsen, A. Checko, and H. L. Christiansen, "A survey of the functional splits in 5g next-generation radio access networks," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2349–2372, 2019.

[5] J. Pérez-Romero, O. Sallent, A. Gelonch, X. Gelabert, B. Klaiqi, M. Kahn, and D. Campoy, "A tutorial on the characterisation and modelling of low layer functional splits for flexible radio access networks in 5g and beyond," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2791–2833, 2023.

[6] X. Foukas *et al.*, "Computational resource consumption of 5g virtualized ran," in *2021 IEEE International Conference on Communications (ICC)*, IEEE, 2021.

[7] O-RAN Alliance, "O-ran fronthaul working group control, user and synchronization plane specification," Tech. Rep. O-RAN.WG4.CUS.0-v11.00, O-RAN Alliance, 2023.

[8] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg, R. S. da Silva, S. Maxenti, L. Bonati, A. Kelkar, C. Dick, E. Baena, J. M. Jornet, T. Melodia, M. Polese, and D. Koutsonikolas, "X5g: An open, programmable, multi-vendor, end-to-end, private 5g o-ran testbed with nvidia arc and openairinterface," *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 11305–11322, 2025.

[9] B. Khan, N. Nidhi, R. R. Paulo, A. Mihovska, and F. J. Velez, "Cost revenue trade-off for the 5g nr small cell network in the sub-6 ghz operating band," in *2022 25th International Symposium on Wireless Personal Multimedia Communications (WPMC)*, pp. 64–69, 2022.

[10] Small Cell Forum (SCF), "5g fapi: Phy api specification," Tech. Rep. SCF222, Small Cell Forum (SCF), 2021.

[11] Small Cell Forum (SCF), "5g network fapi (nfapi) specification," Tech. Rep. SCF225, Small Cell Forum (SCF), 2020.

[12] V. G. Douros, A. Garcia-Saavedra, and C.-P. Xavier, "nfapi: The standard interface for sdn-based 5g small cell networks," *IEEE Communications Standards Magazine*, vol. 2, no. 3, pp. 32–40, 2018.

[13] Small Cell Forum (SCF), "nfapi and fapi specifications," Technical Specification SCF082, Small Cell Forum (SCF), 2021.

[14] S.-Q. Lee and M.-S. Lee, “A method of adaptive low-latency downlink transmission on fapi based 5g nr gnb,” in *2022 13th International Conference on Information and Communication Technology Convergence (ICTC)*, IEEE, 2022.

[15] J. L. Herrera, S. Montebugnoli, D. Scotece, L. Foschini, and P. Bellavista, “A tutorial on O-RAN deployment solutions for 5G: From simulation to emulated and real testbeds,” *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1709–1748, 2026.

[16] Y. Jia, Y. Zhong, M. Wang, J. Gao, P. Zhang, X. Liu, and X. Jin, “Aquifer: Transparent microsecond-scale scheduling for vRAN workloads,” *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3171–3184, 2024.

[17] O. Adamuz-Hinojosa, L. Zanzi, V. Sciancalepore, A. Garcia-Saavedra, and X. Costa-Pérez, “Oranus: Latency-tailored orchestration via stochastic network calculus in 6g o-ran,” in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, pp. 61–70, 2024.

[18] J. Lv, Y. Gao, Z. Ding, Y. Lin, X. You, G. Yang, and W. Dong, “Providing ue-level qos support by joint scheduling and orchestration for 5g vran,” in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, pp. 51–60, 2024.

[19] M. Elfiky, Z. Becvar, and P. Mach, “Allocation of resources for HARQ retransmission in mobile networks based on C-RAN,” *IEEE Systems Journal*, vol. 18, no. 1, pp. 450–461, 2024.

[20] E. S. Hidalgo, J. A. Ayala-Romero, J. X. Salvat Lozano, A. Garcia-Saavedra, and X. C. Perez, “Aegis-RAN: A fair and energy-efficient computing resource allocation framework for vRANs,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 9441–9457, 2025.

[21] L. Diez, A. M. Alba, W. Kellerer, and R. Agüero, “Flexible functional split and fronthaul delay: A queuing-based model,” *IEEE Access*, vol. 9, pp. 151049–151066, 2021.

[22] F. Khan, O. Gil, L. Diez, E. S. Santiago, L. M. Contreras, and R. Agüero, “Evaluating fronthaul network performance under the O-RAN paradigm: A novel methodology based on queuing theory,” *IEEE Transactions on Network and Service Management*, vol. 23, pp. 1723–1741, 2026.

[23] A. S. Mohammed, K. S. Tharakan, H. A. Ammar, H. ElSawy, and H. S. Hassanein, “Fronthaul network planning for hierarchical and radio-stripes-enabled cf-mmimo in o-ran,” *IEEE Transactions on Wireless Communications*, vol. 25, pp. 14781–14796, 2026.

[24] OpenAirInterface Software Alliance, “Oai ran ci/cd scripts and docker framework,” 2024. Accessed: 2024-05-20.

[25] X. Wang *et al.*, “An open-source implementation of the 5g nr functional split option 6,” in *2022 IEEE International Conference on Communications (ICC)*, pp. 1–6, 2022.

[26] V. Jacobson and M. J. Karels, “Congestion avoidance and control,” in *Proceedings of the ACM SIGCOMM Conference on Communications Architectures & Protocols*, pp. 314–329, 1988.

[27] V. Paxson, M. Allman, J. Chu, and M. Sargent, “Computing tcp’s retransmission timer,” RFC 6298, RFC Editor, June 2011.

[28] “Open5gs: Open source 5g core network.” [Online]. Available: https://open5gs.org/, 2024.

[29] OpenAirInterface, “OpenAirInterface Official Website.” [Online]. Available: https://openairinterface.org/, 2025.

[30] “Intel xeon gold 6226r processor (22m cache, 2.90 ghz).” [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/199347/intel-xeon-gold-6226r-processor-22m-cache-2-90-ghz/specifications.html, 2025.

[31] “Broadcom netxtreme bcm5719 quad-port 1gbe base-t adapter.” [Online]. Available: https://www.dell.com/zh-tw/shop/broadcom-5719-%E5%9B%9B%E9%80%A3%E6%8E%A5%E5%9F%A0-1gbe-base-t-%E9%85%8D%E6%8E%A5%E5%8D%A1-pcie-%E5%85%A8%E9%AB%98/apd/540-bdrj/%E7%B6%B2%E8%B7%AF%E8%A8%AD%E5%82%99, 2026.

[32] “Network interface controller.” [Online]. Available: https://en.wikipedia.org/wiki/Network_interface_controller, 2024.

[33] “Intel xeon gold 6433n processor (42m cache, 2.00 ghz).” [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/232148/intel-xeon-gold-6433n-processor-42m-cache-2-00-ghz/specifications.html, 2025.

[34] “Intel ethernet server adapter i210-t1.” [Online]. Available: https://www.intel.com.tw/content/www/tw/zh/products/sku/68668/intel-ethernet-server-adapter-i210t1/specifications.html, 2026.

[35] 3rd Generation Partnership Project (3GPP), *TS 38.101 User Equipment (UE) Radio Transmission and Reception*, Sept. 2024. Version 16.17.0.

[36] O-RAN Working Group 1, *O-RAN Architecture Description*. O-RAN Alliance, June 2025. Rev. 14.00.

[37] O-RAN Working Group 2, *Non-RT RIC Architecture*. O-RAN Alliance, February 2022. O-RAN.WG2.Non-RT-RIC-ARCH-v07.00.

[38] “Android debug bridge (adb) command-line tool.” [Online]. Available: https://developer.android.com/tools/adb, 2024.

[39] ESnet (Energy Sciences Network), “iperf3 - the ultimate speed test tool for tcp, udp and sctp.” [Online]. Available: https://iperf.fr/, 2025.

[40] The Tcpdump Group, “tcpdump.” [Online]. Available: https://www.tcpdump.org/, 2025.

[41] R. Cochran, “Linux ptp v3.1.1: Precision time protocol (ptp) implementation for linux.” [Online]. Available: https://github.com/richardcochran/linuxptp/releases/tag/v3.1.1, 2023.

[42] TSN Community, "Time synchronization - time sensitive networking (tsn)." [Online]. Available: `https://tsn.readthedocs.io/timesync.html#checking-clocks-synchronization`, 2024.

[43] DPDK Project, "Data plane development kit (dpdk) v20.11.9." [Online]. Available: `https://doc.dpdk.org/guides-20.11/`, 2023. Long Term Support (LTS) Release.

[44] N. Nikaein, M. K. Marina, R. Knopp, *et al.*, "Openairinterface: an open lte network in a pc," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 5, pp. 33–40, 2014. Focus on the foundational software architecture of OAI.

[45] Red Hat, "Red hat enterprise linux (rhel)," 2024. Accessed: 2024-05-20.

[46] 3rd Generation Partnership Project (3GPP), *TS 38.212 NR; Multiplexing and channel coding*, 2018. Version 15.0.0. Reference for LDPC, Polar coding validation.

[47] 3rd Generation Partnership Project (3GPP), *TS 38.473 NG-RAN; F1 Application Protocol (F1AP)*, 2020. Version 15.0.0. Reference for Handover Stability and F1-interface.

[48] R. Schmidt, H. Shariatmadari, *et al.*, "Flexric: an sdk for next-generation sd-rans," in *Proceedings of the 17th International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, 2021. Primary reference for E2E 5G SA and RIC Integration.

[49] O-RAN Alliance, "O-ran near-real-time ran intelligent controller architecture & e2 general aspects and principles (e2gap)," tech. rep., O-RAN.WG3.E2GAP-v02.00, 2021.

[50] Advanced Wireless 2 Solutions (AW2S), *AW2S Remote Radio Head (RRH) Integration Guide for 5G NR*, 2022. Reference for third-party RF frontend compatibility.

[51] NVIDIA, "Accelerating 5g vran with nvidia grace hopper and aerial sdk," 2023. Context for RAN-PhySim-GraceHopper-5G.